

Umgang mit dem Kursmaterial

- Sämtliches Kursmaterial (Texte, Folien, Abbildungen, Aufgaben, Lösungen usw.) unterliegt dem Urheberrecht des Dozenten und/oder weiterer Rechteinhaber.
- Die Nutzung ist ausschließlich den Studierenden der angegebenen Fachhochschule im Rahmen des angegebenen Kurses und nur zu Ausbildungszwecken gestattet.
- Jede weitergehende Nutzung, insbesondere das Veröffentlichen, Vervielfältigen, Verbreiten, Teilen oder öffentliche Zugänglichmachen des Materials (z.B. auf Webseiten, in Cloud-Speichern, per E-Mail, in Präsentationen oder Publikationen), ist ohne ausdrückliche schriftliche Zustimmung untersagt und kann rechtliche Konsequenzen nach sich ziehen.
- Die Nutzung des Kursmaterials in Verbindung mit Sprachmodellen oder KI-Systemen (z.B. LLMs wie ChatGPT) ist nur dann zulässig, wenn sichergestellt ist, dass die Inhalte nicht zum Training, zur Speicherung oder zu einer sonstigen Weiterverarbeitung durch Dritte verwendet werden (wie z.B. dem LLM-Anbieter) und alle oben genannten Nutzungsbeschränkungen eingehalten werden.

Was ist ein Dateisystem (File System)?





Kapitel_04

Dateisysteme

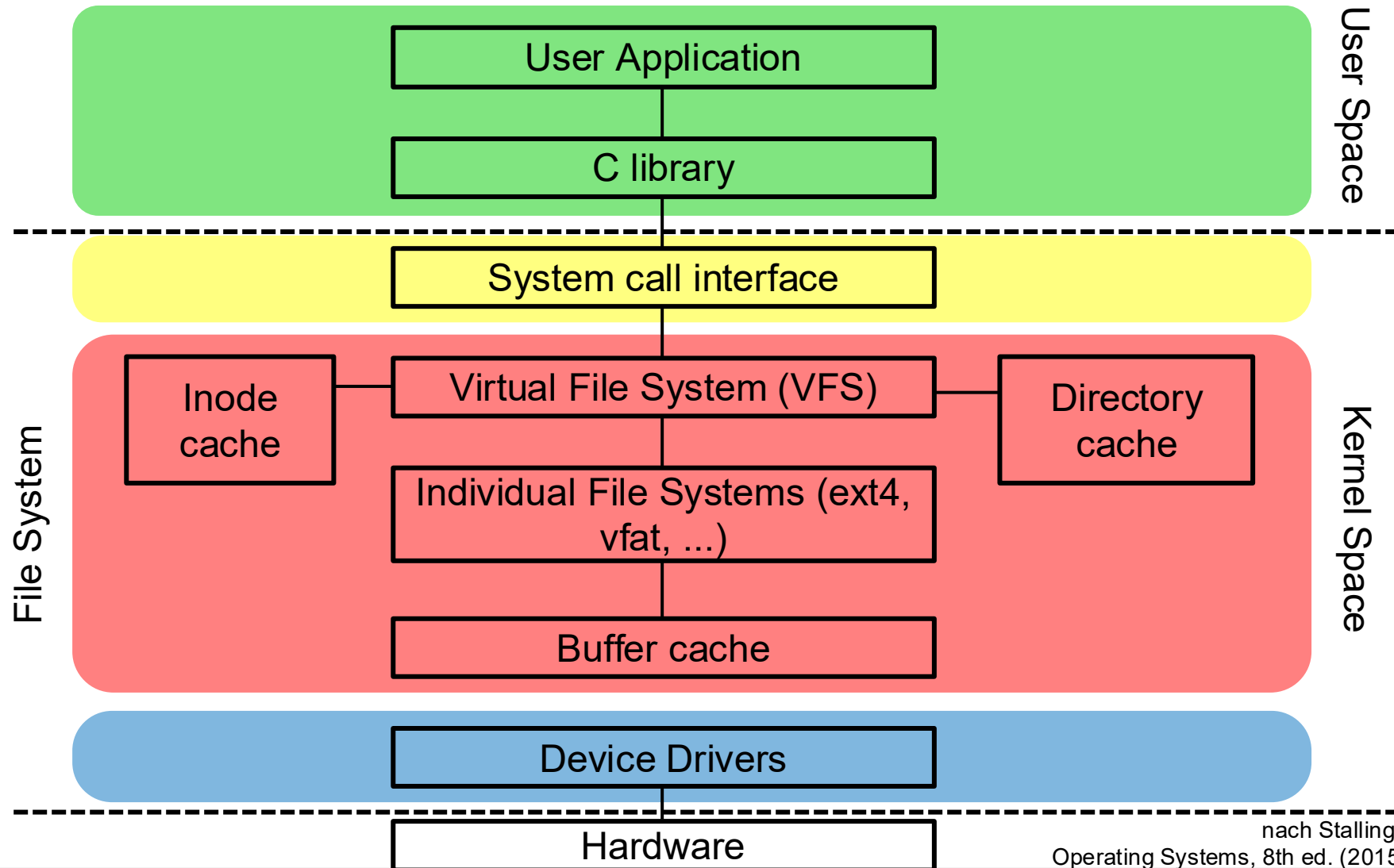
Betriebssysteme

Prof. Dr. Claus Fühner
Fakultät Informatik

Ostfalia Hochschule für angewandte Wissenschaften
Braunschweig/Wolfenbüttel



Übersicht Architektur Dateisysteme (Linux)

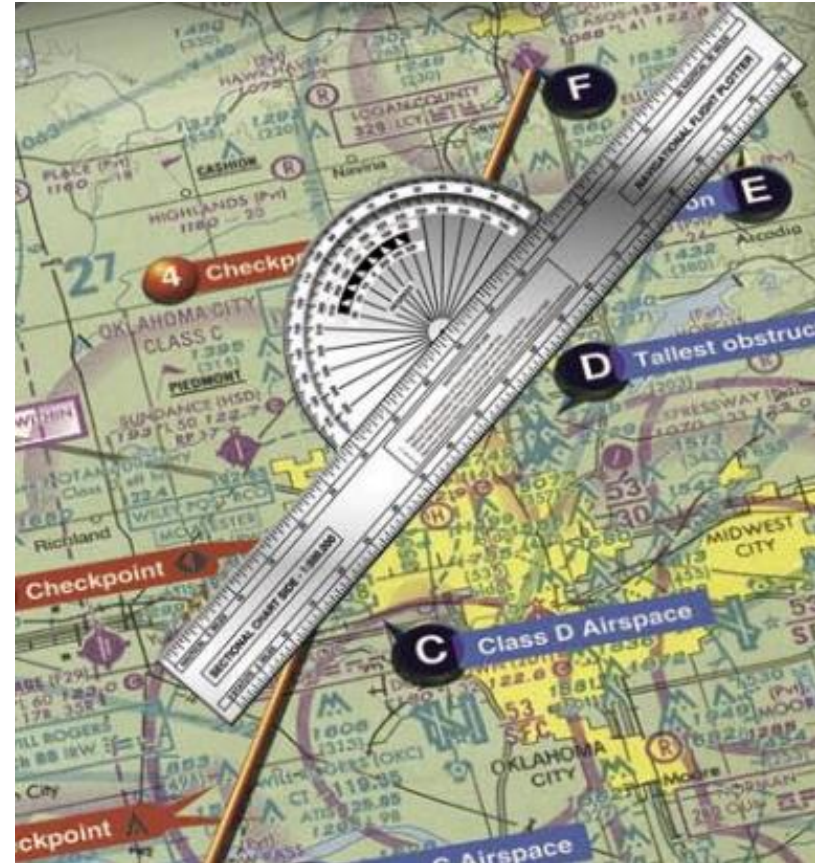


Agenda



Dateisysteme

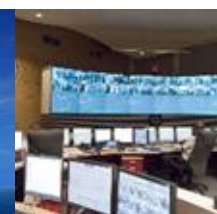
- Grundkonzepte der Implementierung
- Ablegen von Dateien in Blöcken
- Umgang mit freien Blöcken
- Verwaltung von Verzeichnissen
- Beispiele
 - FAT
 - UNIX
- Festplatten, SSDs und RAID
- Moderne Konzepte
- Praxis-Probleme



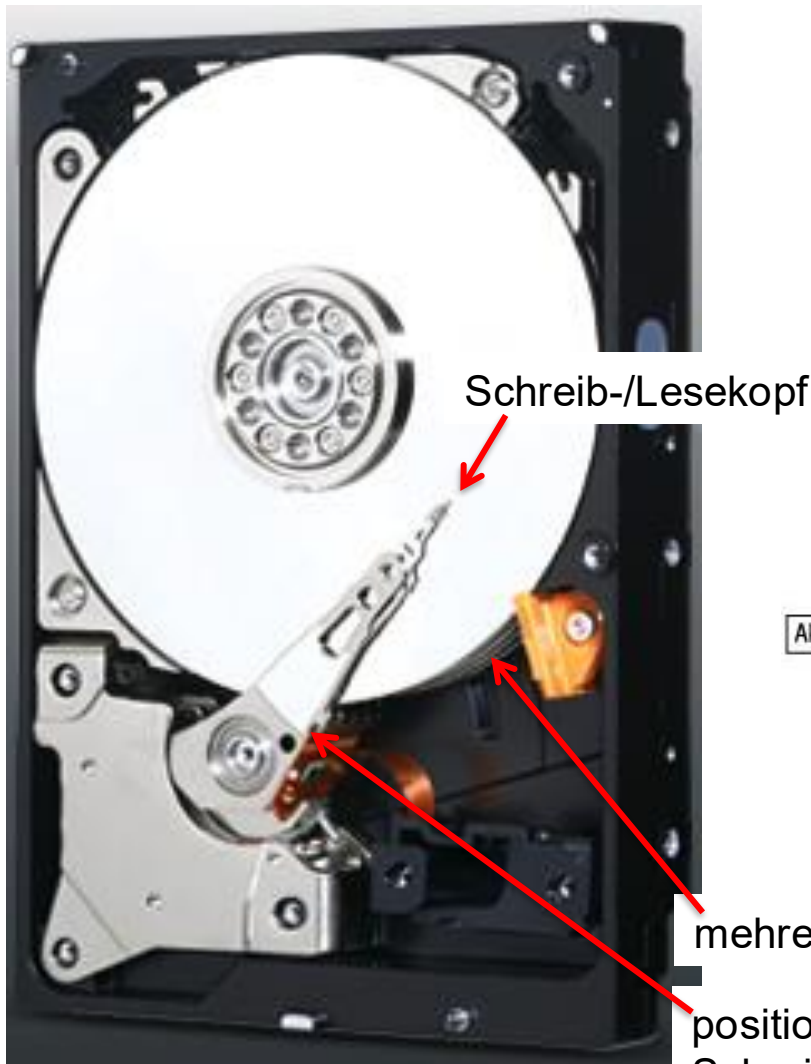
Nur zu Lehrzwecken



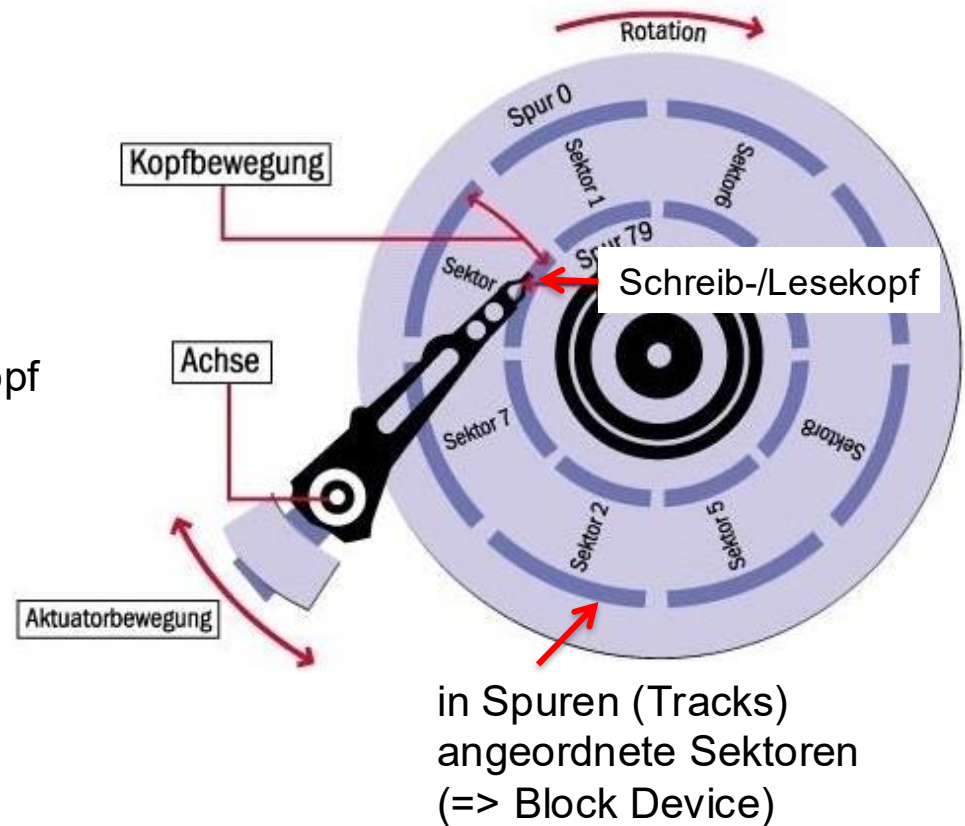
Betriebssysteme



Speichern von Dateien in Blöcken



Bildquelle: Western Digital



Bildquelle: http://www.tecchannel.de/bild-zoom/401602/5/325117/IM_56F45DD4-F1F5-85AA-9CE088B43FF06FE3/

Realisierung von Dateien

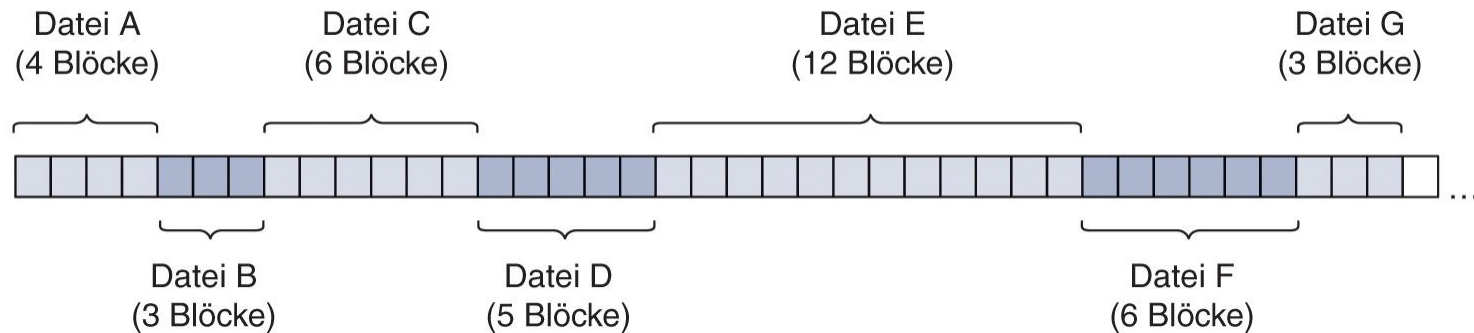
- Fragestellung: Wie ordnen wir die einzelnen Blöcke (z. B. 4 kB) auf einem Datenträger den einer (längeren) Datei zu?
- Ansätze:
 - Zusammenhängende Belegung
 - Verkettete Listen
 - Index-Nodes („I-Nodes“)
- Kriterien:
 - Möglichst wenig **Fragmentierung** durch Löschen und Größenänderung von Dateien
 - **Effizienz** im Zugriff (wahlfreier Zugriff möglich)
 - **Speichereffizienz** (kompakte Verwaltungsstrukturen)

Sektoren, Blöcke und Cluster

- **Sektoren** auf dem Solid State Drive sind fortlaufend nummeriert
- Gilt mit **Logical Block Addressing (LBA)** auch für Festplatten: Platte rechnet fortlaufende Nummer intern in **Kopf-, Spur- und Sektornummer** um.
- Betriebssystem fasst mehrere Sektoren zu **Clustern** zusammen, um Aufwand in der Verwaltung zu sparen. Die Größe ist einstellbar.
- Widersprüchliche Begriffsdefinition für **Block**
 - **Windows:**
 - **Block** wird häufig als Begriff für **Sektor** verwendet
 - **Cluster** („Zuordnungseinheit“) – bezeichnet ein Vielfaches eines Sektors, d.h. ein Vielfaches von z.B. 512 Byte oder 4 kiB.
 - **Unix:**
 - **Block** ist ein Vielfaches eines **Sektors**.
- Cluster-/Blockgröße wird beim Anlegen eines Dateisystems (**Formatieren**) festgelegt und ist wichtig für die **Effizienz** des Dateisystems

Variante 1: Zusammenhängende Belegung

- Dateien werden **hintereinander** abgelegt



- Löschen/Größenänderung von Dateien möglich? **Fragmentierung?**
- Effizienz **wahlfreier Zugriff** (seek)?
- **Speichereffizienz**/Verwaltungsoverhead?
- Weitere Vor-/Nachteile?

Einschub: Fragmentierung

Interne Fragmentierung

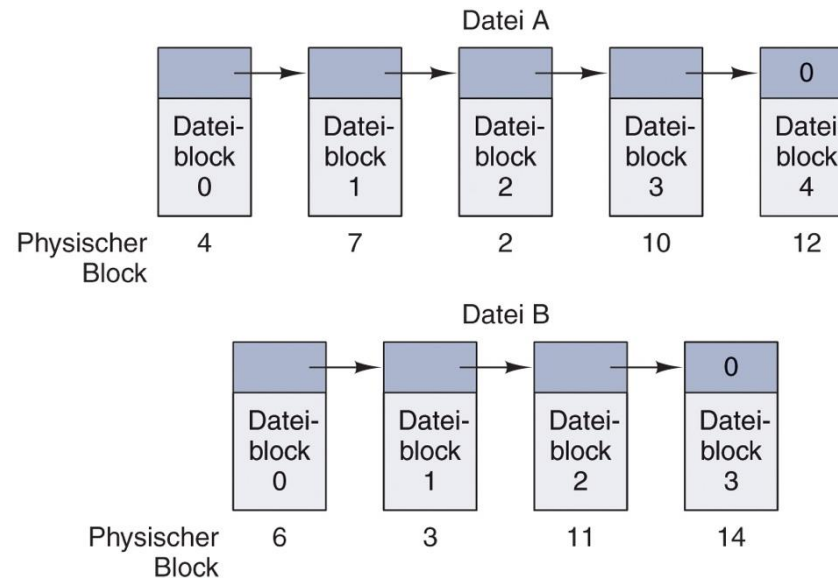
- Belegung nur von ganzen Blöcken => letzter Block nur teilweise genutzt
- Ineffizienz steigt mit der Block-/Clustergröße
 - Macht sehr große Blöcke/Cluster besonders für kleine Dateien ineffizient, für sehr große Dateien aber nicht so relevant
- Beispiel: Clustergröße 32 kiB, Verschnitt in Prozent bei Dateigröße
 - 1 kiB
 - $99 * 32 \text{ kiB} + 1 \text{ kiB}$

Externe Fragmentierung

- Blöcke einer Datei nicht "am Stück" gespeichert, sondern verstreut
 - Kopf einer Festplatte muss sich dann häufig bewegen → defragmentieren
 - Externe Fragmentierung für SSDs nicht relevant
- **Externe Fragmentierung** auch für **freien Speicher** relevant:
genug Speicher frei, aber nicht „am Stück“

Variante 2: Verkettete Liste

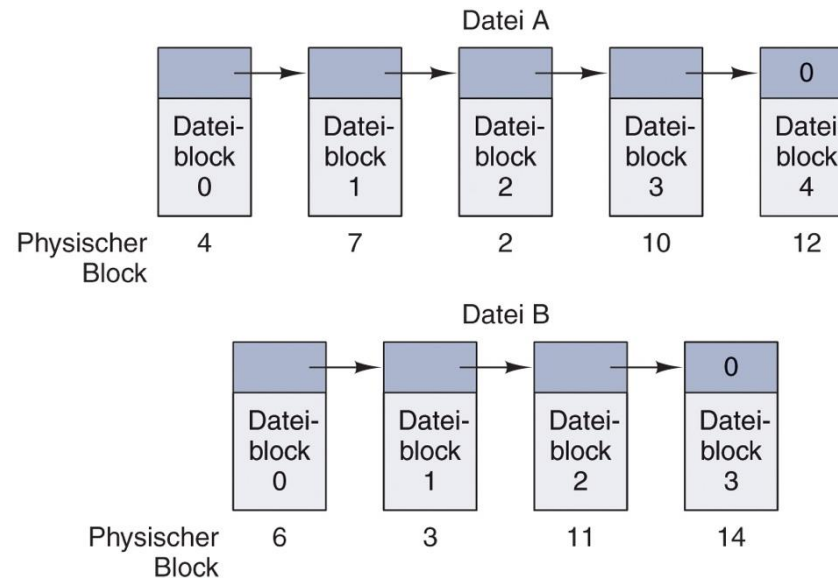
- Jede Datei wird durch eine **verkettete Liste** von Plattenblöcken dargestellt



- Löschen/Größenänderung** von Dateien möglich? **Fragmentierung**?
- Effizienz **wahlfreier Zugriff** (seek)?
- Speichereffizienz**/Verwaltungsoverhead?
- Weitere Vor-/Nachteile?

Variante 2: Verkettete Liste

- Jede Datei wird durch eine **verkettete Liste** von Plattenblöcken dargestellt



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A			2		0			1			3		4			
B				1			0					2			3	

(Externe) Fragmentierung des freien Speichers und der Daten!!!

Variante 3: Index-Nodes („I-Nodes“)

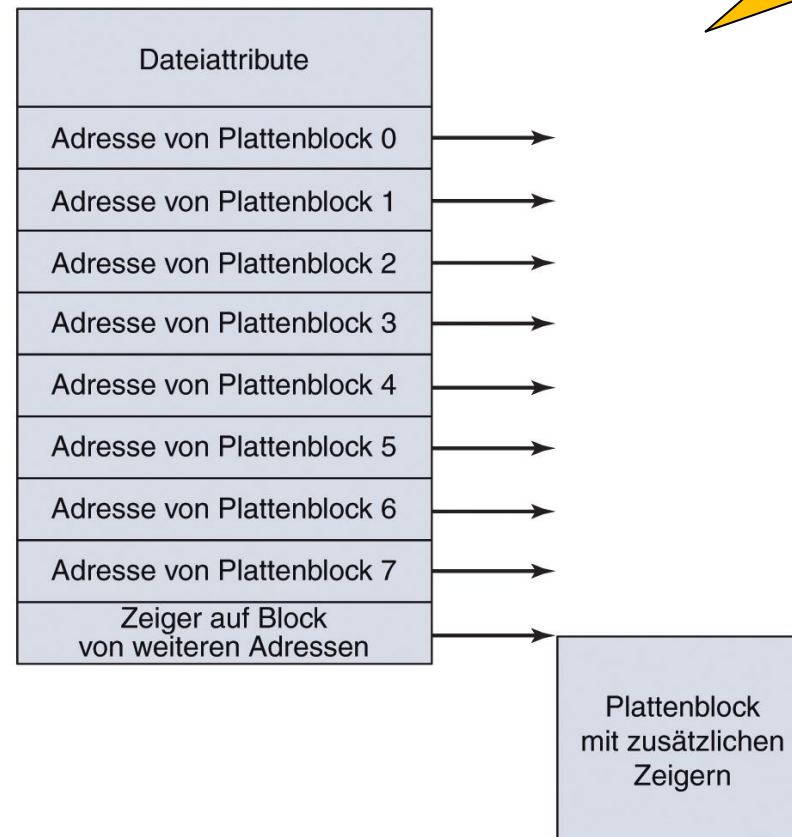
- Für jede Datei einen Index-Knoten (I-Node) anlegen
- Dieser enthält die Nummern der Plattenblöcke; I-Node mit 8 Adressen:

- Löschen/Größenänderung von Dateien möglich? Fragmentierung?
- Effizienz wahlfreier Zugriff (seek)?
- Speichereffizienz/Verwaltungsoverhead?
- Große Dateien? Weitere Vor-/Nachteile?



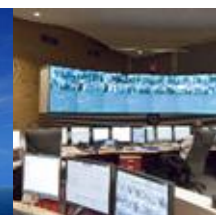
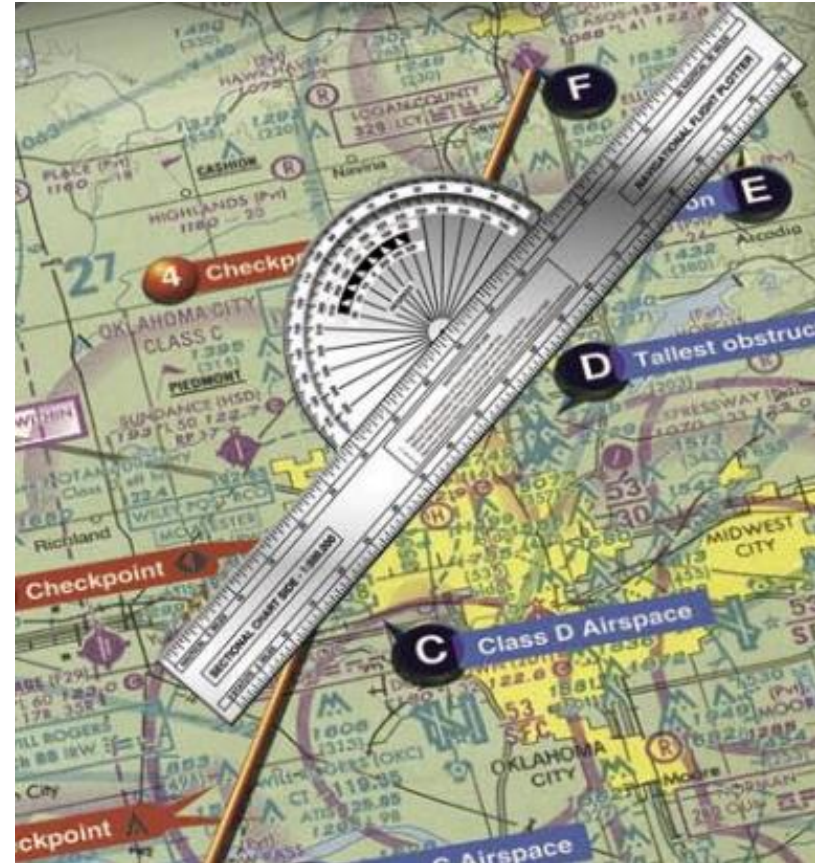
Variante 3: Index-Nodes („I-Nodes“)

Idee: jeder Datei wird
ein I-Node zugeordnet



Dateisysteme

- Grundkonzepte der Implementierung
 - Ablegen von Dateien in Blöcken
 - Umgang mit freien Blöcken
 - Verwaltung von Verzeichnissen
- Beispiele
 - FAT
 - UNIX
- Festplatten, SSDs und RAID
- Moderne Konzepte
- Praxis-Probleme



Anforderungen

Freie Blöcke finden

- Zum Schreiben neuer Blöcke müssen wir sehr schnell **freie Blöcke finden**.
- Rekonstruktion **anhand der** durch Dateien **belegten Blöcke** viel zu aufwändig

Zusammenhängende Belegung

- Zum **Schreiben** großer Datenmengen ist es **effizienter**, diese in einen zusammenhängenden freien Bereich zu schreiben.
- Auch **Lesezugriffe** und ggf. sogar die Verwaltung werden dann **effizienter**.

Lösungsvarianten

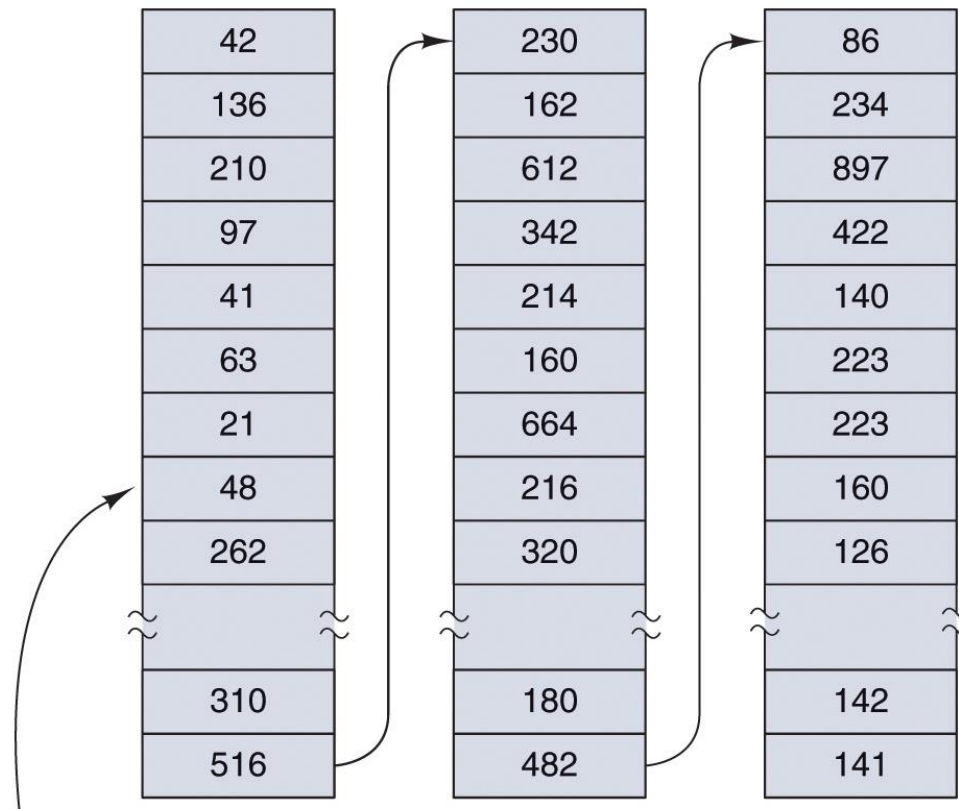
1. Bitmap

2. Freibereichsliste: Verkettete Liste der freien Blöcke

In beiden Fällen speichern wir **zusätzliche (redundante) Information!**

➔ Zusätzliche Anforderung: Schnelle Aktualisierung der neuen Datenstruktur

Variante 1: Freibereichsliste/-tabelle



Ein 1-KB-Plattenblock kann 256
32-Bit-Plattenblocknummern speichern

Freibereichsliste (ähnlich Index-Nodes)

- Plattenblöcke **enthalten Nummern aller freien Blöcke**
- Letzter Eintrag in Block: Blocknummer adressiert weiteren Block, der weitere Nummern freier Blöcke enthält
- Diese Liste kann in ansonsten freien Blöcken gespeichert werden

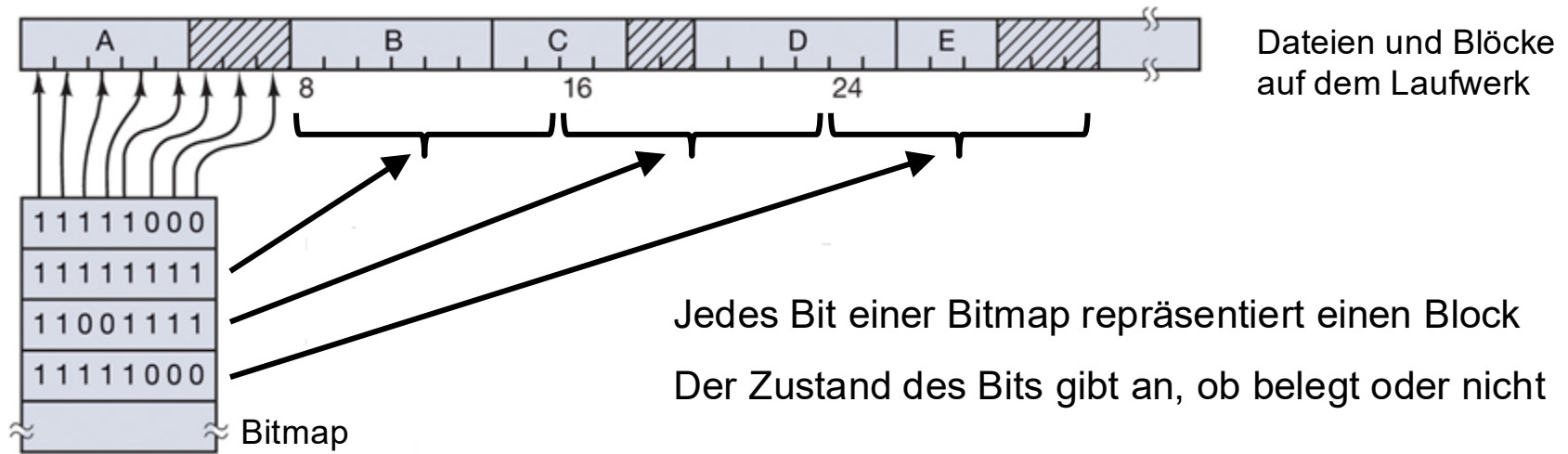
Variante 2: Bitmap

- Die Bits in einer Bitmap sind fest Blöcken zugeordnet und geben den Belegungsstatus an.
- Beispiel: Für freie Blöcke ist das Bit in der Bitmap 0, für belegte Blöcke 1

Dateien und Blöcke
auf dem Laufwerk

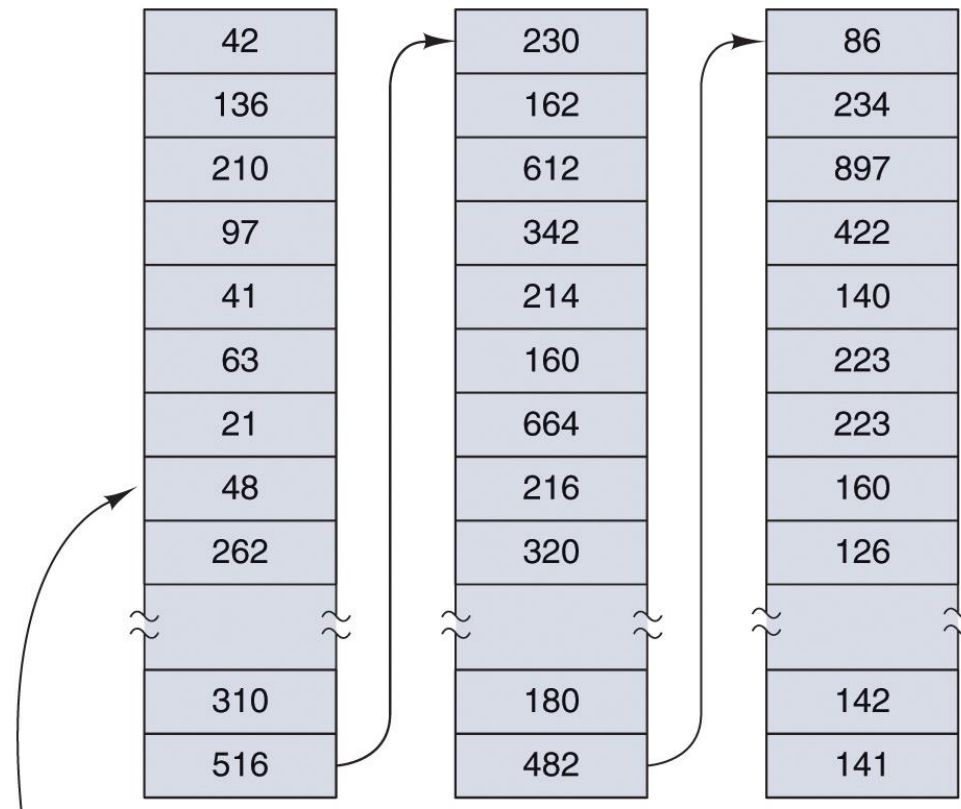
Variante 2: Bitmap

- Die Bits in einer Bitmap sind fest Blöcken zugeordnet und geben den Belegungsstatus an.
- Beispiel: Für freie Blöcke ist das Bit in der Bitmap 0, für belegte Blöcke 1



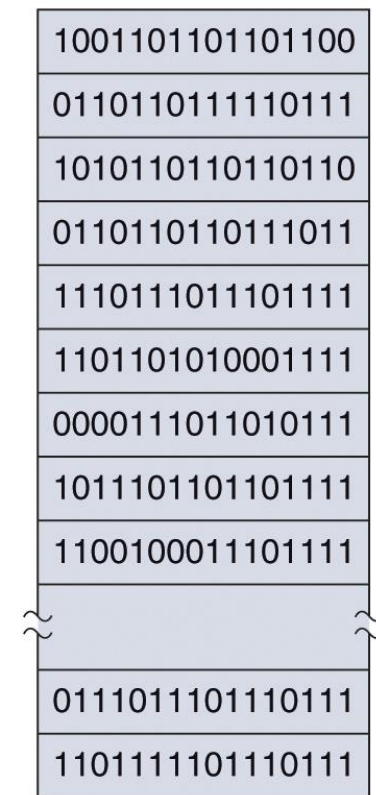
- *Weiterer Trade-off Blockgröße:*
 - kleine Blöcke → große Bitmap, die viel Speicherplatz erfordert (Verwaltungsoverhead)
 - große Blöcke → viel Verlust durch interne Fragmentierung

Varianten im Vergleich



Ein 1-KB-Plattenblock kann 256
32-Bit-Plattenblocknummern speichern

Freibereichsliste (ähnlich Index-Nodes)

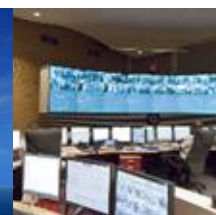
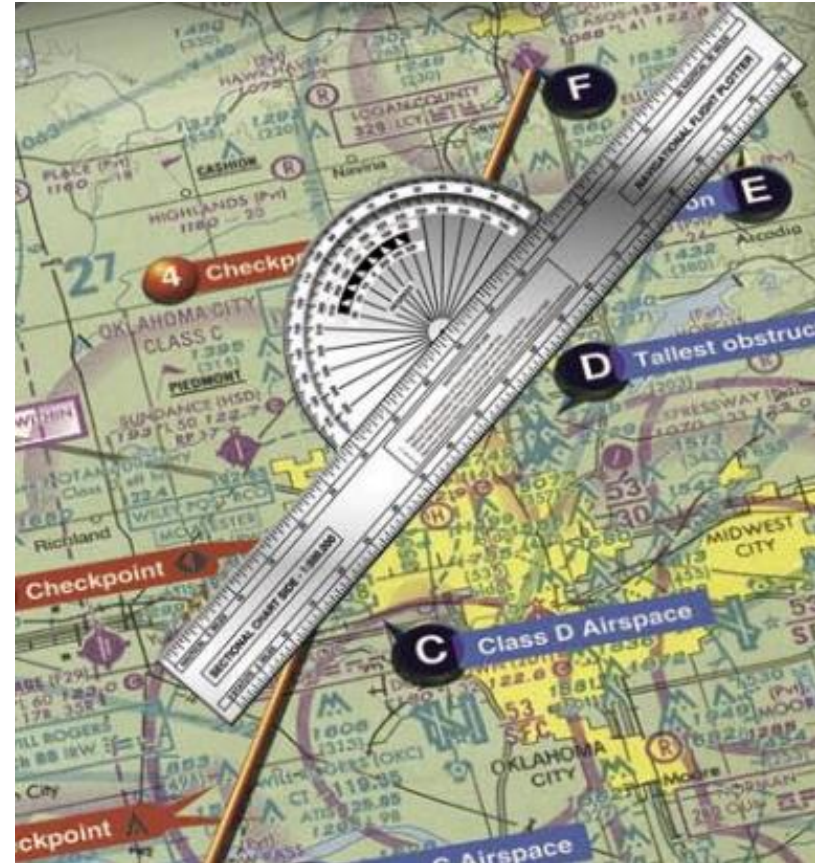


Eine Bitmap

Bitmap

Dateisysteme

- Grundkonzepte der Implementierung
 - Ablegen von Dateien in Blöcken
 - Umgang mit freien Blöcken
- ➔ • Verwaltung von Verzeichnissen
- Beispiele
 - FAT
 - UNIX
- Festplatten, SSDs und RAID
- Moderne Konzepte
- Praxis-Probleme



Metadaten / Dateiattribute

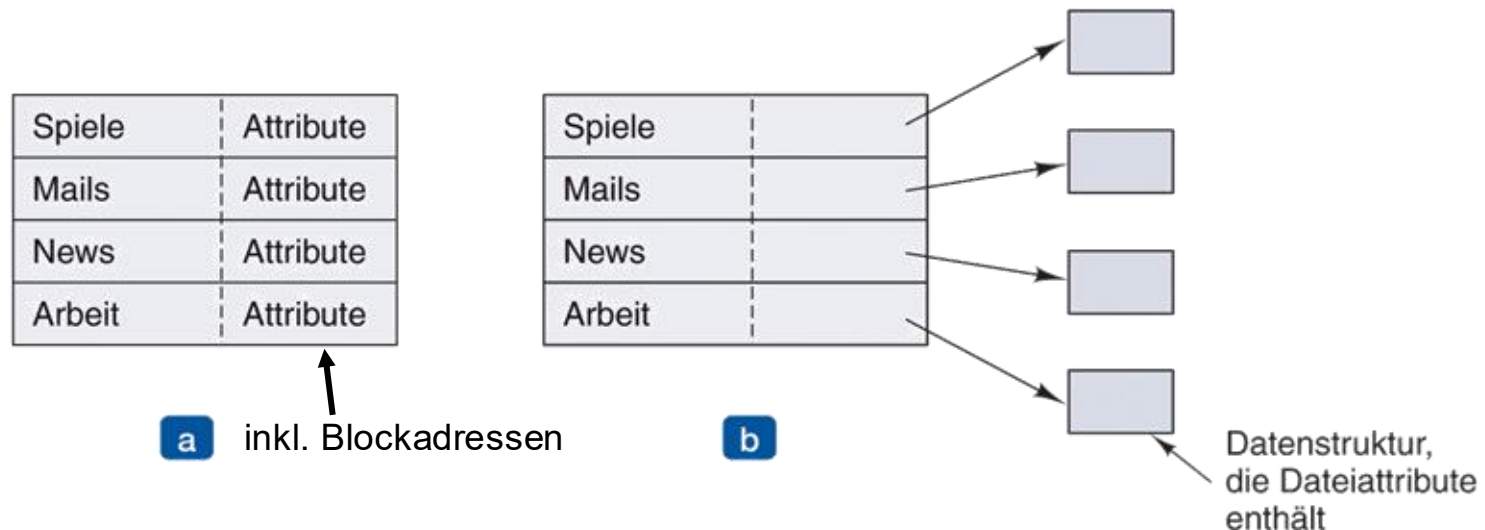
Was müssen wir außer den eigentlichen Nutzdaten für jede Datei noch speichern?

- Dateiname
- Datum und Uhrzeit von
 - Erstellung
 - letzte Änderung
 - letzter Zugriff
- Besitzerin
- Zugriffsrechte für Besitzer und andere User
- Weitere Attribute
 - Nur Lesen
 - Nicht löschen
 - Versteckte Datei
 - Datei wurde in diesem Zustand archiviert
 - ...
- Details siehe Anhang!

Konkrete Metadaten hängen vom Dateisystem ab!

Realisierung von Verzeichnissen

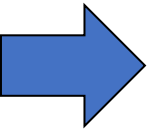
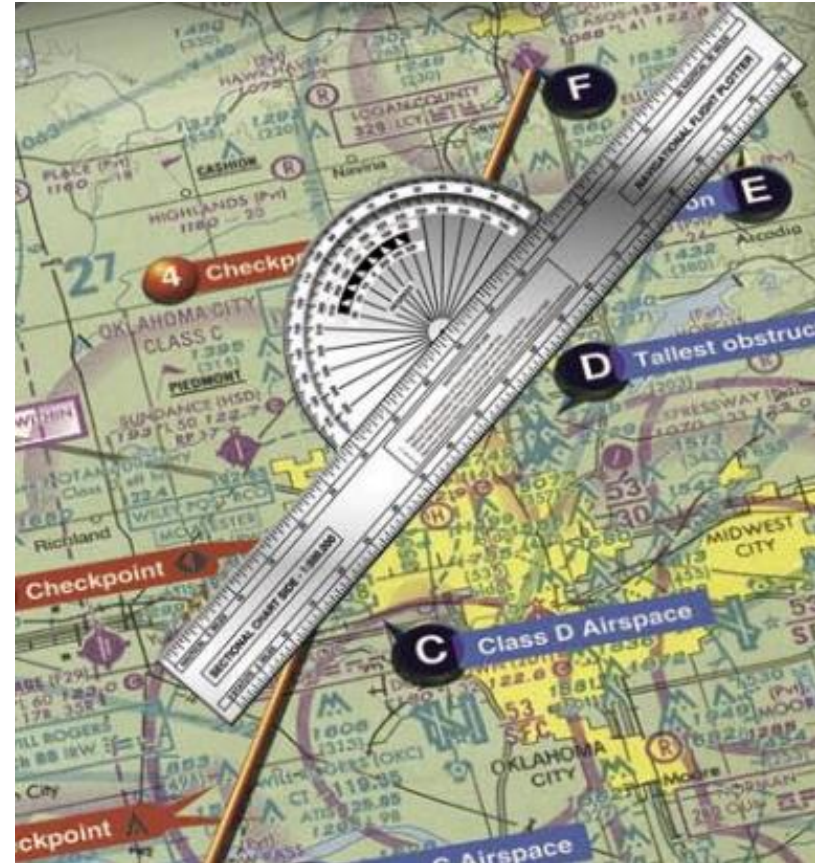
- Verzeichnisse sind **Tabellen (Arrays)** mit je einem Eintrag pro enthaltener Datei (eigentliche Daten in der Abb. nicht gezeigt!)



- Speichern der **Dateiattribute** in:
 - a) Einfachem Verzeichnis mit Attributen inkl. Blockadresse(n)
 - b) Verzeichnis mit Dateinamen und Verweis auf I-Nodes (Attribute+Blkadr)
- Wurzelverzeichnis wird an einem fixen Ort gespeichert
- Unterverzeichnisse werden wie Dateien behandelt

Dateisysteme

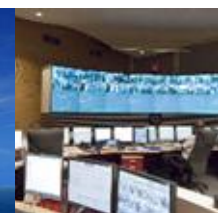
- Grundkonzepte der Implementierung
 - Ablegen von Dateien in Blöcken
 - Umgang mit freien Blöcken
 - Verwaltung von Verzeichnissen
- Beispiele
 - FAT
 - UNIX
- Festplatten, SSDs und RAID
- Moderne Konzepte
- Praxis-Probleme



Nur zu Lehrzwecken



Betriebssysteme



File Allocation Table (FAT)

- Wurde von Microsoft entwickelt und in **MS-DOS** ab 1981 publiziert
- Hierarchisches Dateisystem (ab MS-DOS 2.0)
- Kein Benutzerkonzept (Einbenutzersystem)
- MS-DOS nicht für parallele Prozesse ausgelegt (es läuft nur ein Programm)
- Nutzung einer **File Allocation Table** (FAT)
- FAT-32 und Nachfolger **ExFAT** von allen gängigen Betriebssystemen, Kameras und Handys (Speicherkarten) unterstützt (!)
 - **FAT-32** auch für Systemlaufwerke älterer Windows-Versionen, heute hauptsächlich als Austauschformat
 - Nachfolger ExFAT als Austauschformat auf externen Datenträgern (Festplatten, SSDs, Speicherkarten)

Plattenlayout FAT-Dateisystem

- Logisches Laufwerk in 6 Bereiche aufgeteilt:

Bereich 1	Bereich 2	Bereich 3	Bereich 4	Bereich 5	Bereich 6
Bootsektor	Reservierte Sektoren	FAT1	FAT2	Stammverzeichnis	Daten

Plattenlayout FAT-Dateisystem

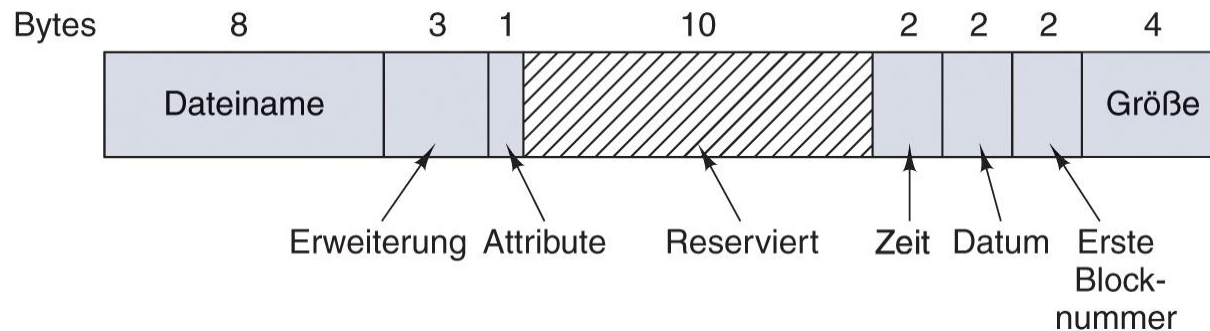
- Logisches Laufwerk in 6 Bereiche aufgeteilt:

Bereich 1	Bereich 2	Bereich 3	Bereich 4	Bereich 5	Bereich 6
Bootsektor	Reservierte Sektoren	FAT1	FAT2	Stammverzeichnis	Daten

- Bootsektor**: enthält x86-Maschinencode zum Starten des Betriebssystems sowie Infos zum Layout wie Blockgröße, Clustergröße, Anzahl Blöcke, etc.
- Reservierte Sektoren** z.B. für Bootmanager
- Zentrale Datenstruktur **FAT** von entscheidender Bedeutung, deshalb auch als Kopie gespeichert (FAT1 + FAT2)
- Stammverzeichnis** (Root Directory) mit fester Größe (-> maximale Anzahl Verzeichniseinträge für FAT-12/FAT-16, in FAT-32 anders gelöst)
- ... dann folgen die Cluster mit den eigentlichen **Daten** (bei weitem größter Bereich)

Verzeichniseintrag – Ursprungsversion 1981

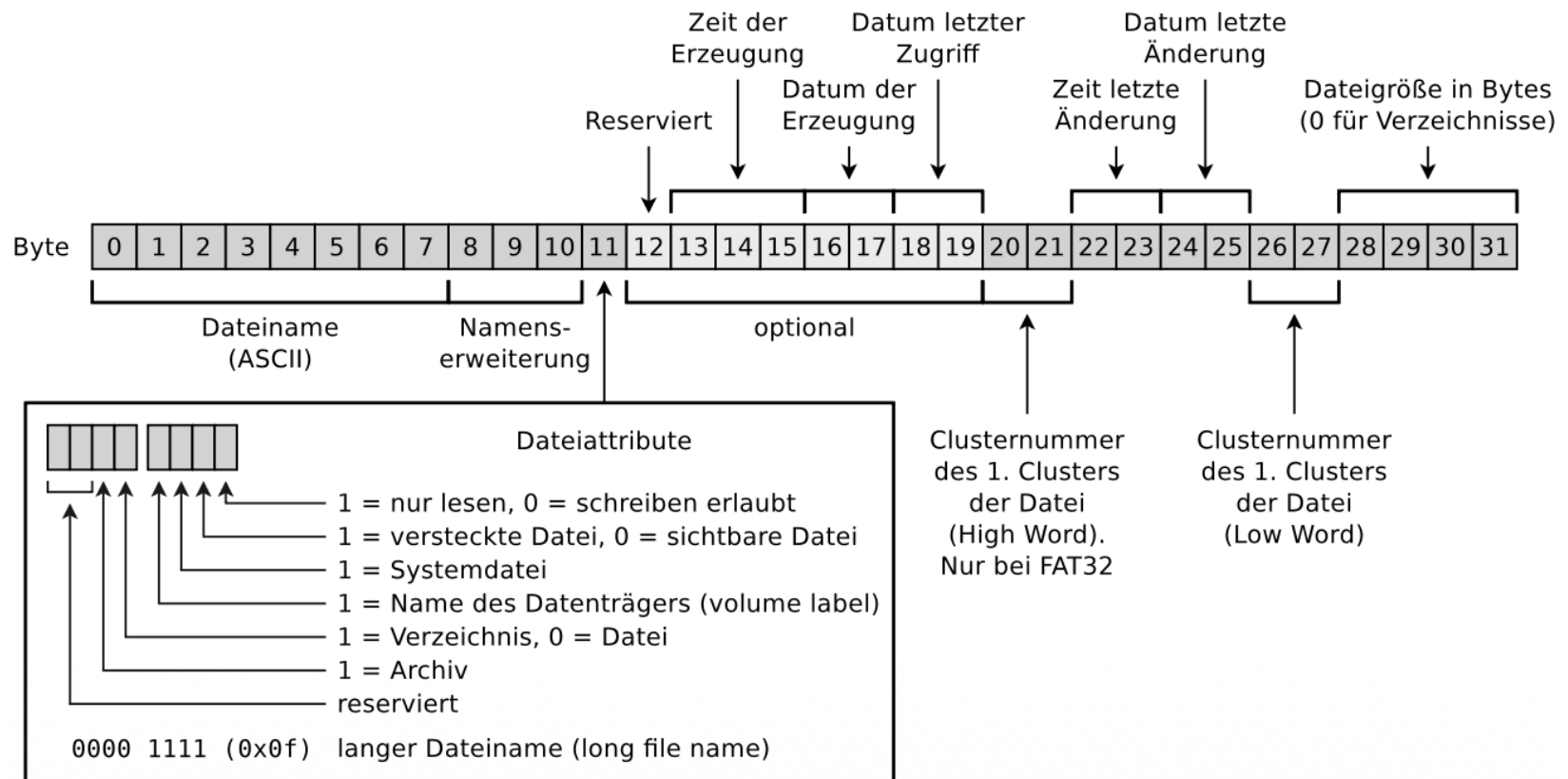
- Unterstützt nur 8.3 Dateinamen (ab Windows 95/VFAT auch länger, ➔Anhang)
- MS-DOS-Verzeichniseintrag mit 32 Byte:



- Gelöschte Dateien werden durch 0xE5 im ersten Zeichen des Dateinamens markiert; ansonsten bleibt der Verzeichniseintrag erhalten!

Verzeichniseintrag – weiterentwickelte Version

- Unterstützt nur 8.3 Dateinamen (ab Windows 95/VFAT auch länger, ➔Anhang)



File Allocation Table (FAT)

Besondere Werte in FAT
Kettenende (im Beispiel -1)
freie Cluster (im Beispiel -2, in realen FATs 0)

Verzeichnis

Dateiname	Attribute	Block 1
Datei_A	...	4
Datei_B	...	6
- leer -		
- leer -		

FAT

0	
1	
2	10
3	11
4	7
5	
6	3
7	2
8	
9	
10	12
11	14
12	-1
13	
14	-1
15	

Plattenblöcke mit eigentl. Daten (2 B)

3. →	0: ab
	1: cd
	2: ef
1. →	3: gh
	4: ij
	5: kl
2. →	6: mn
	7: op
	8: qr
	9: st
4. →	10: uv
	11: wx
5. →	12: yz
	13: 01
	14: 23
	...

FAT kombiniert Verkettungsinformation mit Management freier Cluster

Beispiel: FAT Dateisystem

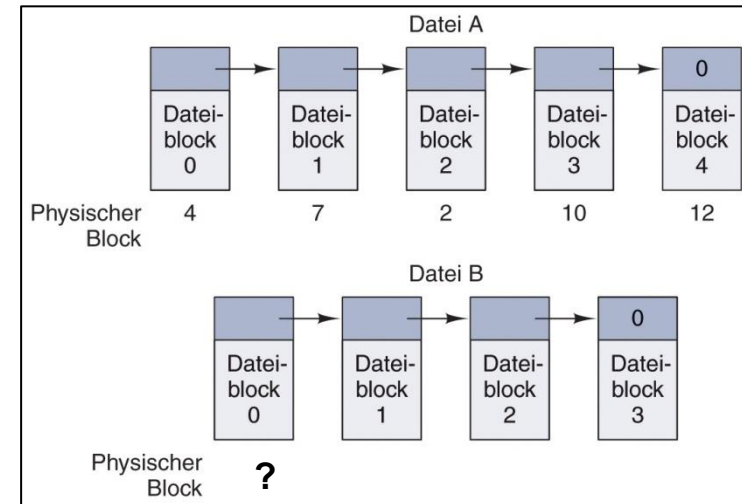
File Allocation Table (FAT)

Verzeichnis

Dateiname	Attribute	Block 1
Datei_A	...	4
Datei_B	...	6
- leer -		
- leer -		

FAT

0	
1	
2	10
3	11
4	7
5	
6	3
7	2
8	
9	
10	12
11	14
12	-1
13	
14	-1
15	



ähnlich verkettete Liste

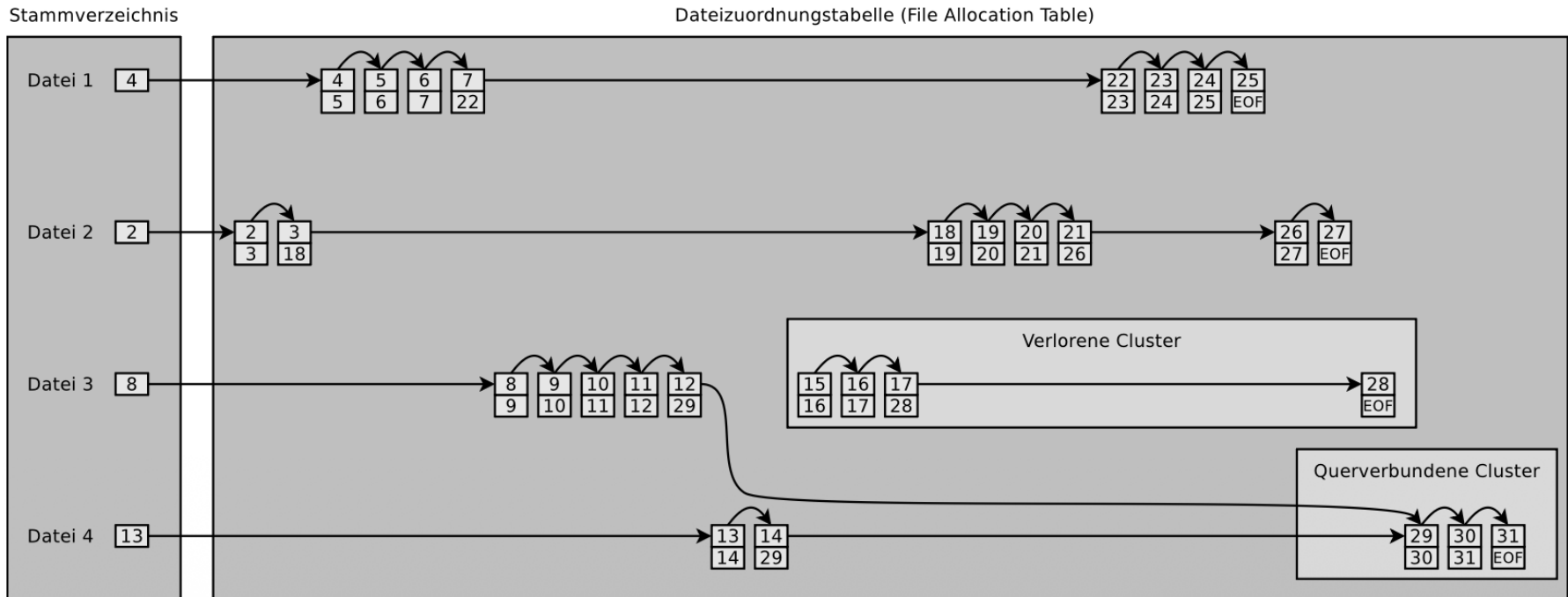
Varianten FAT(-12), FAT-16, FAT-32

- **FAT-12**, **-16** und **-32** stehen für die Anzahl der Bits, die in der FAT für eine Blockadresse verwendet werden
- FAT-32 Besonderheiten
 - verwendet nur 28 Bit für Blockadresse (!)
 - Root Directory / Stammverzeichnis flexibler Größe im Datenbereich
 - maximale Dateigröße 4 GiB - 1 Byte
- Maximale Größe des Dateisystems:

Block Size	FAT-12	FAT-16	FAT-32
0.5 KB	2 MiB		
1 KB	4 MiB		
2 KB	8 MiB	128 MiB	
4 KB	16 MiB	256 MiB	1 TiB
8 KB		512 MiB	2 TiB
16 KB		1024 MiB	4 (2) TiB
32 KB		2048 MiB	8 (2) TiB

Der MBR beschränkt die Partitionsgröße bei *partitionierten* Medien auf max. 2 TiB. Dann ist nur noch *eine* Partition möglich (siehe Anhang)!

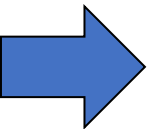
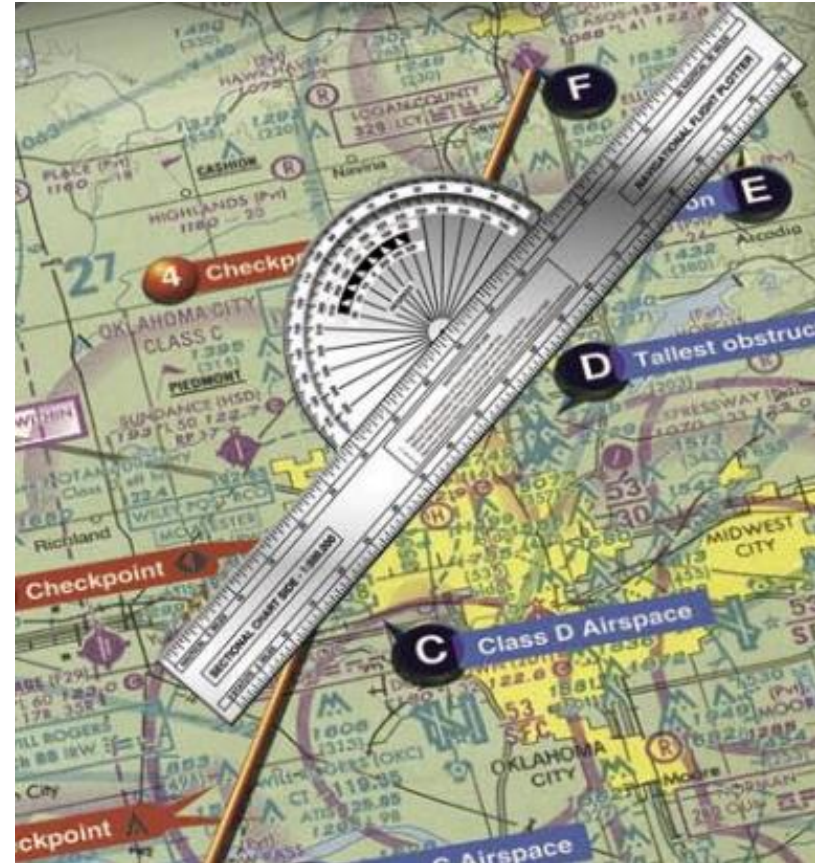
Gefahr von Inkonsistenzen



- **Verlorene Cluster:** Kette 15,16,17,28 gehörte zu Datei 3, ist durch Fehler verloren gegangen
- **Querverbundene Cluster:** Kette 29,30,31 aus Datei 4 ist fehlerhaft zusätzlich in Datei 3 gelandet

Dateisysteme

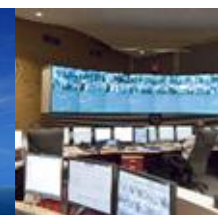
- Grundkonzepte der Implementierung
 - Ablegen von Dateien in Blöcken
 - Umgang mit freien Blöcken
 - Verwaltung von Verzeichnissen
- Beispiele
 - FAT
 - UNIX
- Festplatten, SSDs und RAID
- Moderne Konzepte
- Praxis-Probleme



Nur zu Lehrzwecken



Betriebssysteme



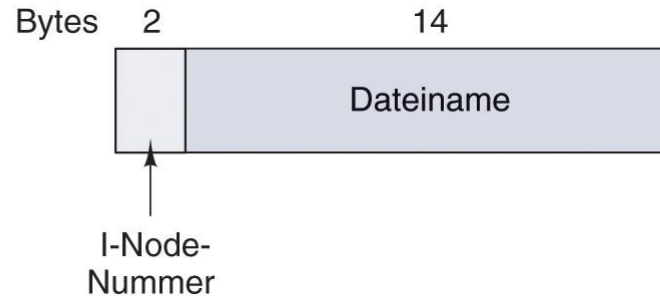
Plattenlayout Minix-Dateisystem

- Minix-Dateisystem aus dem Buch Operating Systems von A. S. Tanenbaum
- Standard-Dateisystem für die allerersten Linux-Versionen
- Ein logisches Laufwerk wird in folgende 6 Bereiche aufgeteilt

Bereich 1	Bereich 2	Bereich 3	Bereich 4	Bereich 5	Bereich 6
Bootblock (1 Cluster)	Superblock (1 Cluster)	Inodes-Bitmap (1 Cluster)	Cluster-Bitmap (1 Cluster)	Inodes (15 Cluster)	Daten (Restliche Cluster)

- **Bootblock:** enthält den Bootloader zum Laden des Kernels
- **Superblock:** enthält Infos über Layout, Anzahl I-Nodes, Anzahl Blöcke, etc.
- I-Nodes Bitmap: Frei (0) / Belegt (1) – Bitmap für alle I-Nodes
- Cluster-Bitmap: Frei (0) / Belegt (1) – Bitmap für alle Cluster (Blöcke)
- **I-Nodes** Metadaten (64 Byte) für genau eine Datei oder Verzeichnis
 - Ein I-Node enthält (u.a.) die Nummern der Cluster mit den eigentlichen Dateiinhalten.

UNIX-V7-Verzeichniseintrag



- UNIX Version 7 (PDP-11), 1979
- Dateinamen bis zu 14 Zeichen lang
- Beliebige Zeichen ohne / und NUL
- Anzahl Dateien pro Dateisystem $< 2^{16} = 64\text{ K}$
- Einträge werden unsortiert als Array gespeichert (bis einschl. ext2)
- Mehrere Verzeichniseinträge können auf den gleichen I-Node verweisen (Hard-Links)

Tanenbaum: Moderne Betriebssysteme, Pearson (2025).

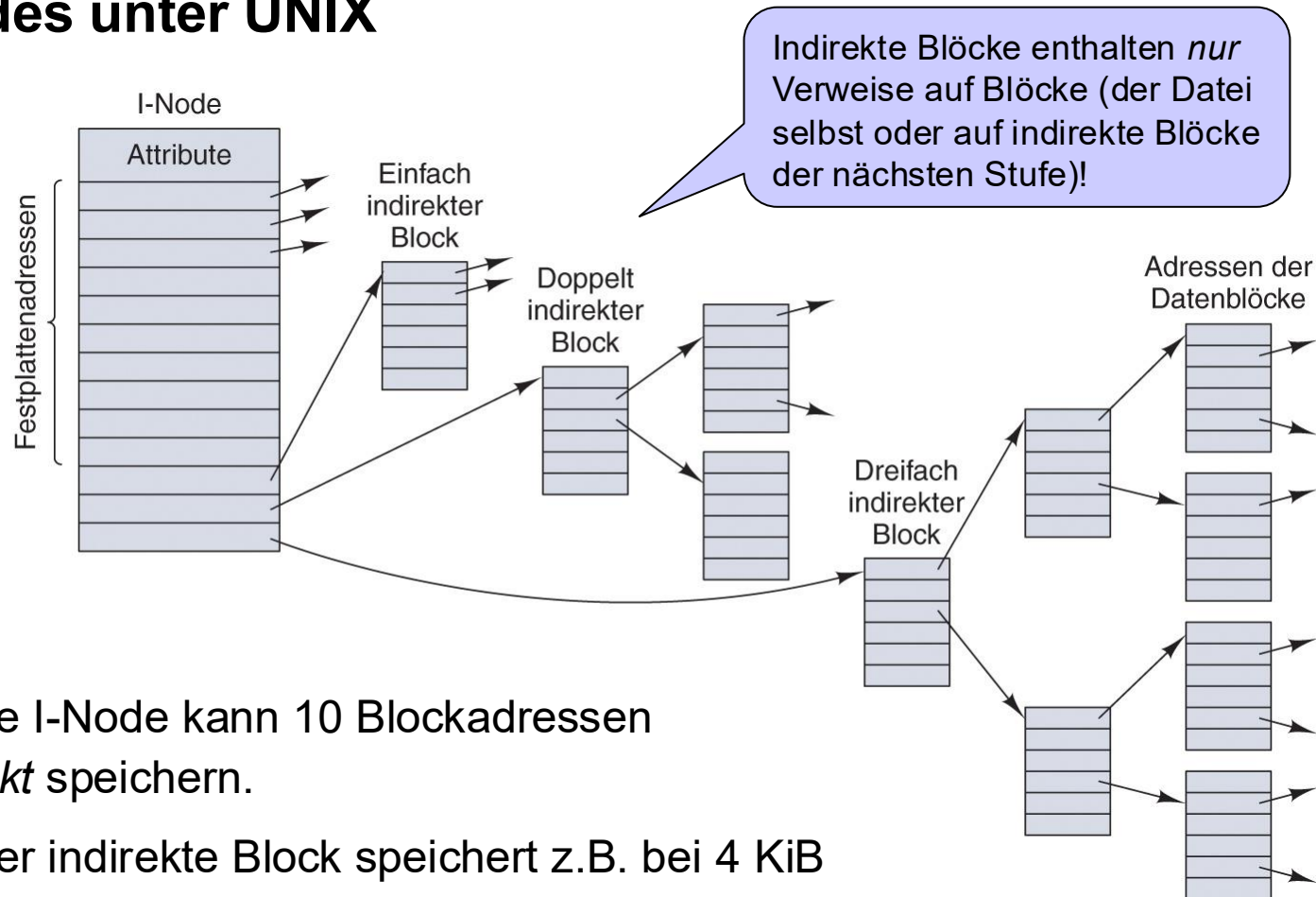
Struktur der I-Nodes unter System V

- I-Nodes bestehen aus Attributen:

Field	Bytes	Description
Mode	2	File type, protection bits, setuid, setgid bits
Nlinks	2	Number of directory entries pointing to this i-node
Uid	2	UID of the file owner
Gid	2	GID of the file owner
Size	4	File size in bytes
Addr	39	Address of first 10 disk blocks, then 3 indirect blocks
Gen	1	Generation number (incremented every time i-node is reused)
Atime	4	Time the file was last accessed
Mtime	4	Time the file was last modified
Ctime	4	Time the i-node was last changed (except the other times)

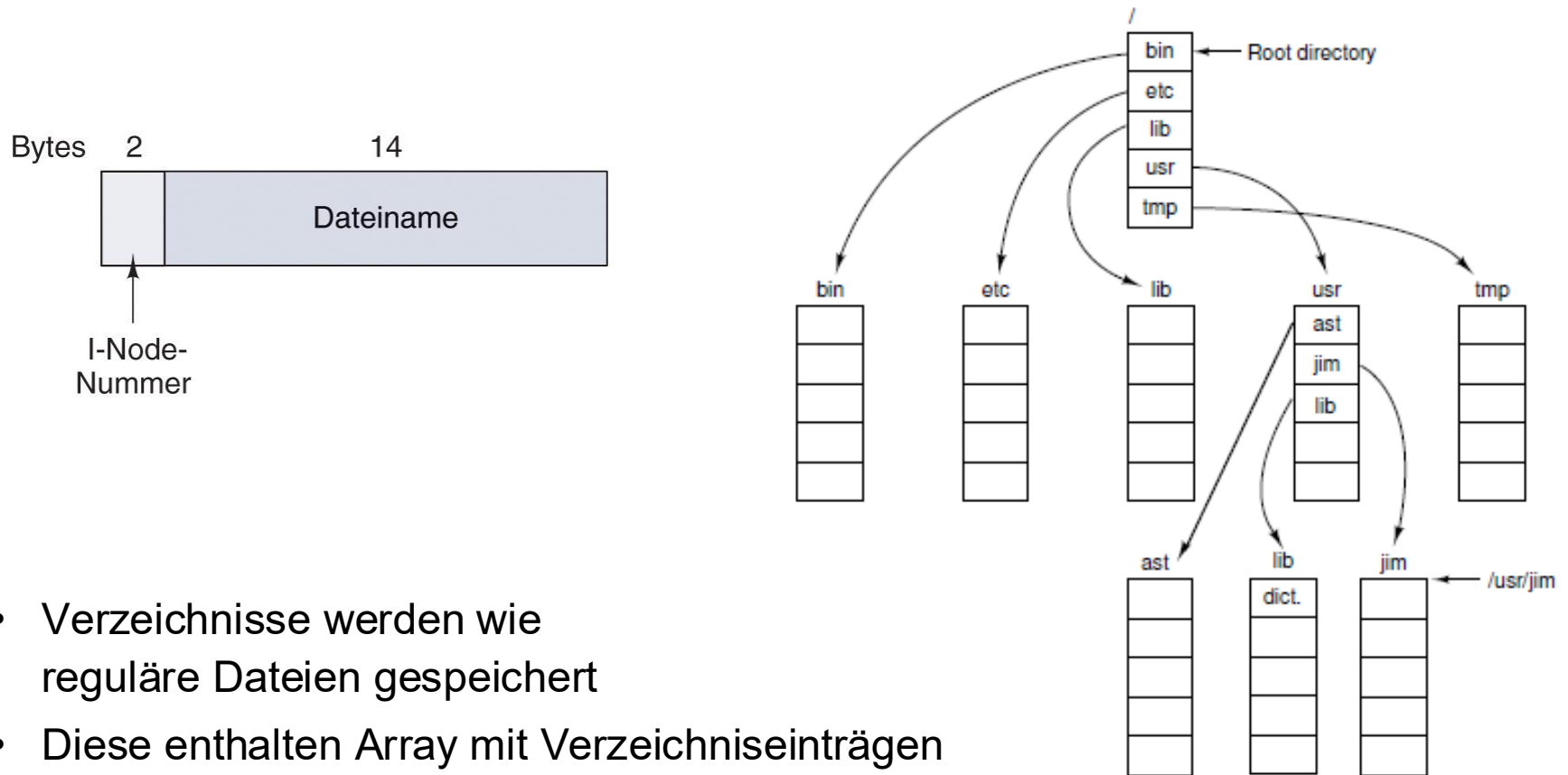
- ... und 13 Feldern für Blockadressen

I-Nodes unter UNIX



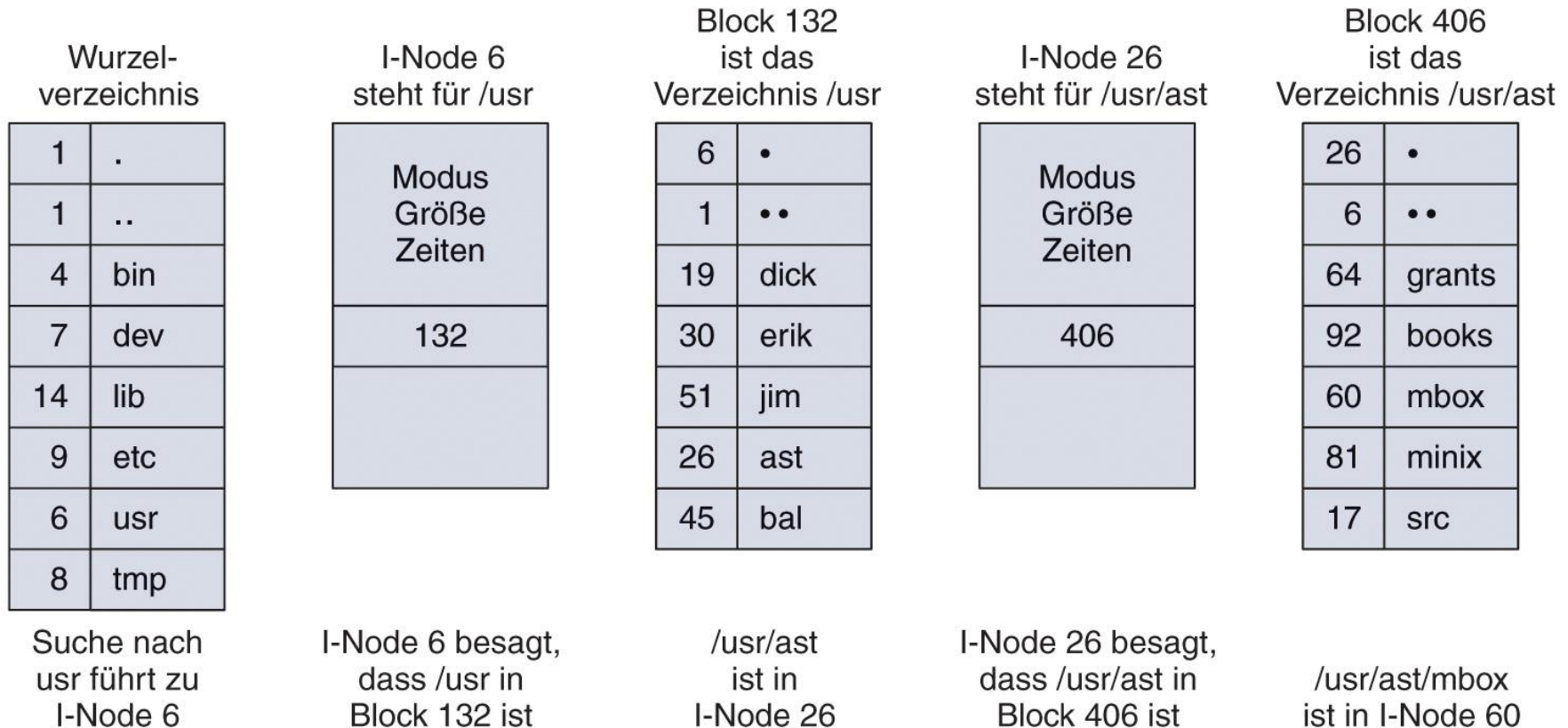
- Jede I-Node kann 10 Blockadressen *direkt* speichern.
- Jeder indirekte Block speichert z.B. bei 4 KiB Blockgröße und 4 Byte pro Blockadresse *1024 Blockadressen*
- Wie groß kann eine Datei maximal werden? → Übung

UNIX-Verzeichnisbaum /usr/ast/mbox



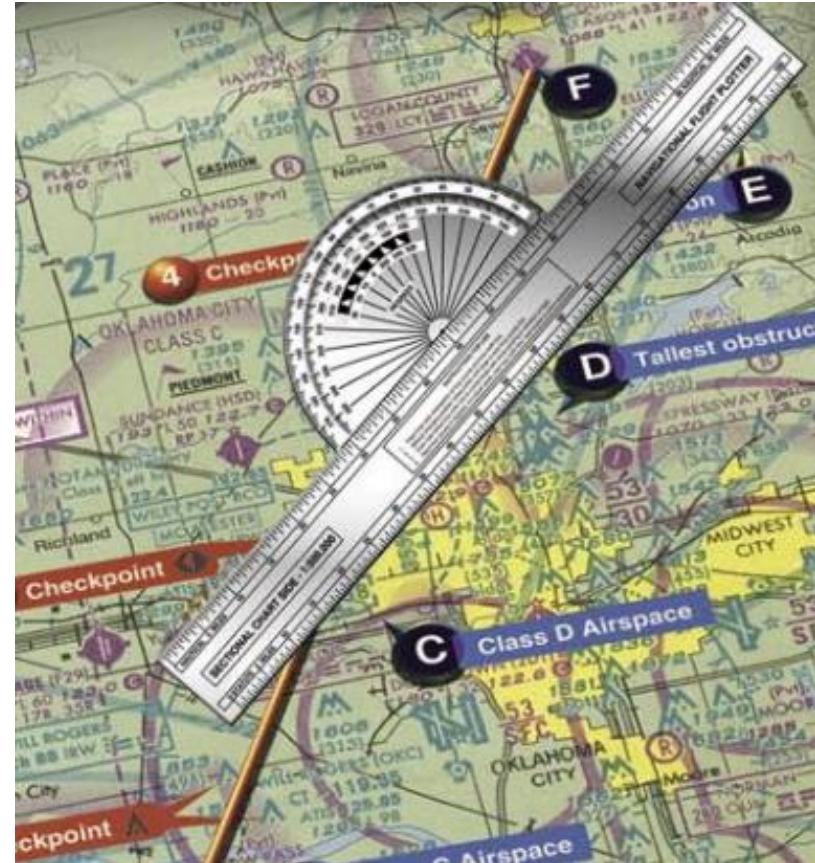
- Verzeichnisse werden wie reguläre Dateien gespeichert
- Diese enthalten Array mit Verzeichniseinträgen (nicht sortiert)
- Wurzelverzeichnis (Root Directory) ist in I-Node 2 zu finden (fixer Wert)
- I-Node 1 verwaltet eine Datei mit *Bad Blocks* und I-Node 0 wird nicht genutzt

I-Nodes /usr/ast/mbox



Dateisysteme

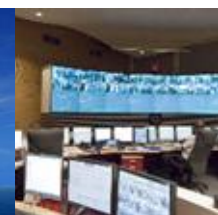
- Grundkonzepte der Implementierung
 - Ablegen von Dateien in Blöcken
 - Umgang mit freien Blöcken
 - Verwaltung von Verzeichnissen
- Beispiele
 - FAT
 - UNIX
- ➔ Festplatten, SSDs und RAID
 - Moderne Konzepte
 - Praxis-Probleme

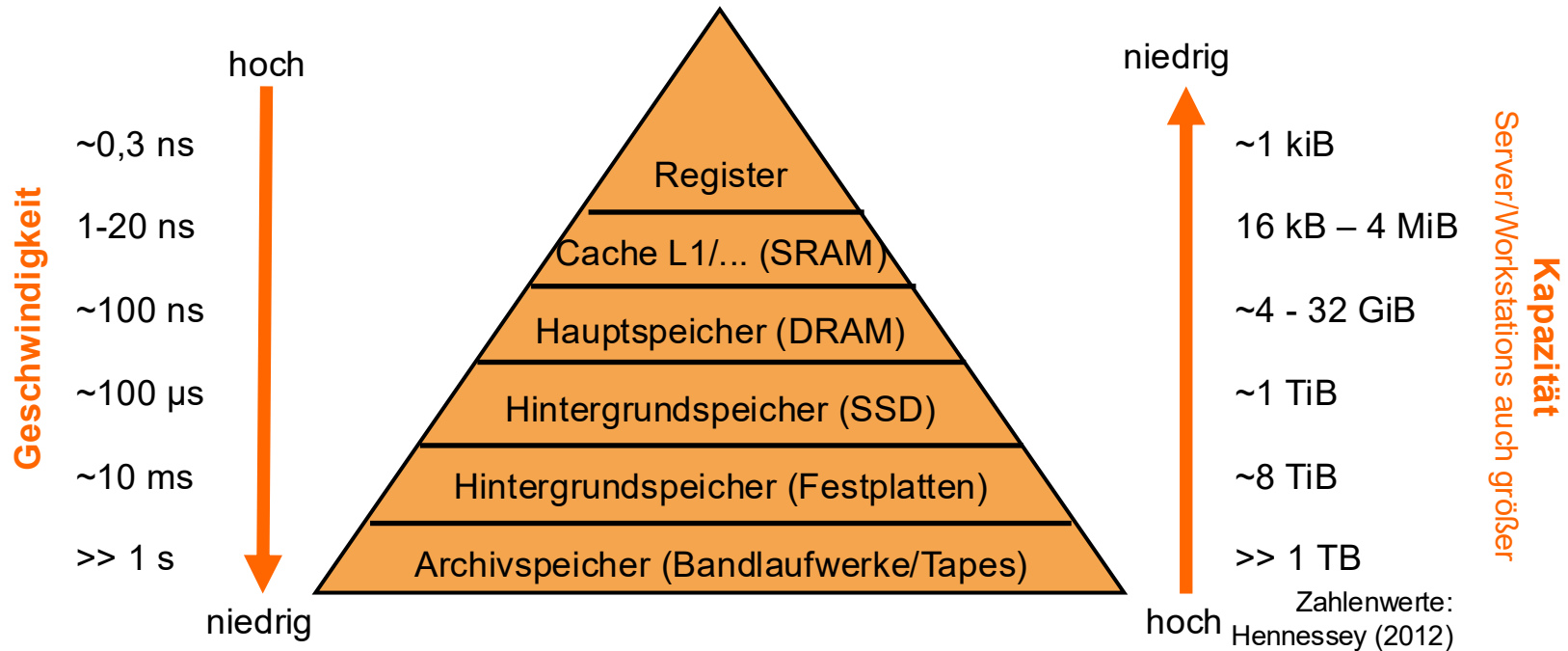


Nur zu Lehrzwecken



Betriebssysteme

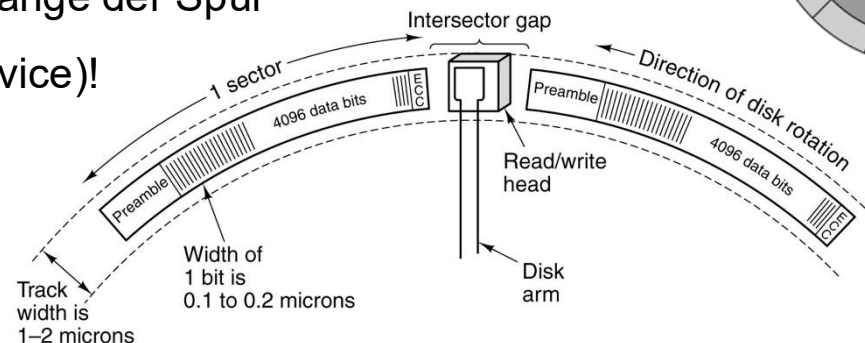
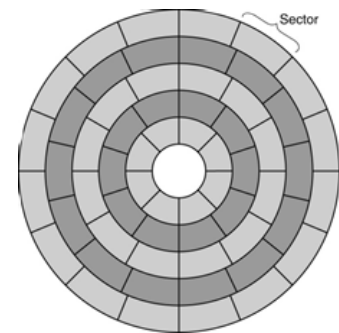
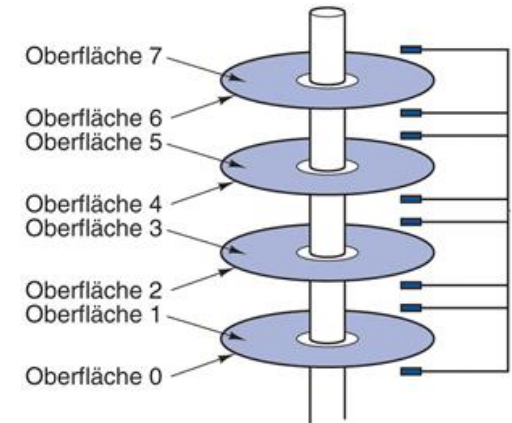




- RAM ist ca. **100'000x so schnell** wie **Festplatten** und **1000x so teuer**
- RAM ist ca. **1'000x so schnell** wie **NVMe-SSD** und **100x so teuer**
- typische Speicherkapazitäten resultieren aus dem Preis für das Speichermedium
- Also:
 - selten genutzte Daten auf günstigen Speichermedien halten
 - häufig genutzte Daten in schnellen (teuren) Speichermedien halten / cachen

Physischer Aufbau und Organisation der Daten

- Eine Festplatte besteht aus mehreren **Disks (Scheiben)**, meist Nutzung beider **Oberflächen**
- Scheibendurchmesser meist **2,5 Zoll oder 3,5 Zoll**
- Disks drehen mit konstanter, hoher Geschwindigkeit (ca. **3'000-15'000 Umdrehungen/Minute**)
- Oberfläche aufgeteilt in kreisförmigen **Tracks (Spuren)**
- Die Arme mit den Schreib-/Leseköpfen (Kopfträger) können nur gemeinsam bewegt werden auf **Cylinder (Zylinder)** übereinander liegender Tracks
- Track aufgeteilt in **Sektoren / Blöcke**; Größe fest, aber unterschiedliche Anzahl je nach Länge der Spur
- Zugriff nur Sektorweise (Block Device)!



Zugriffszeit

- **Beispiel:** Seagate Barracuda 7200.12
Formfaktor 3,5“, SATA-Interface, 4 Köpfe/2 Scheiben,
1 TB, 32 MB Cache, 7200 U/min, ca. 2009
- **Mittlere Zugriffszeit:** Latenzzeit für den Zugriff auf den ersten Sektor
Summe aus:
 - **Controller overhead:** Verarbeitungszeit der Plattenelektronik 12,9 ms
 - **Seek time:** Zeit für den Spurwechsel, also das Anfahren des gewünschten Zylinders abhängig von der Leistungsfähigkeit des Kopfträger-Antriebes 0,2 ms
 - **Rotations-Latenzzeit:** mittlere Zeit, bis sich die Platte soweit gedreht hat, dass sich der gesuchte Sektor unter den Schreib-/Lesekopf befindet abhängig von der Drehgeschwindigkeit (1/2 Umdrehung bei 120 U/s) 8,5 ms
- **Kontinuierliche Datenrate:** Durchsatz für das Lesen (Sequential Read) 4,2 ms
- **I/O Operations Per Second (IOPS)** 125 MiB/s
- *für Lesen kurzer, zufällig ausgewählter 4 kiB Blöcke (Random Access)* 75 IOPS
- *IOPS beim kontinuierlichen Lesen, Blockgröße 4 kiB (Sequential Read)* 32 k IOPS

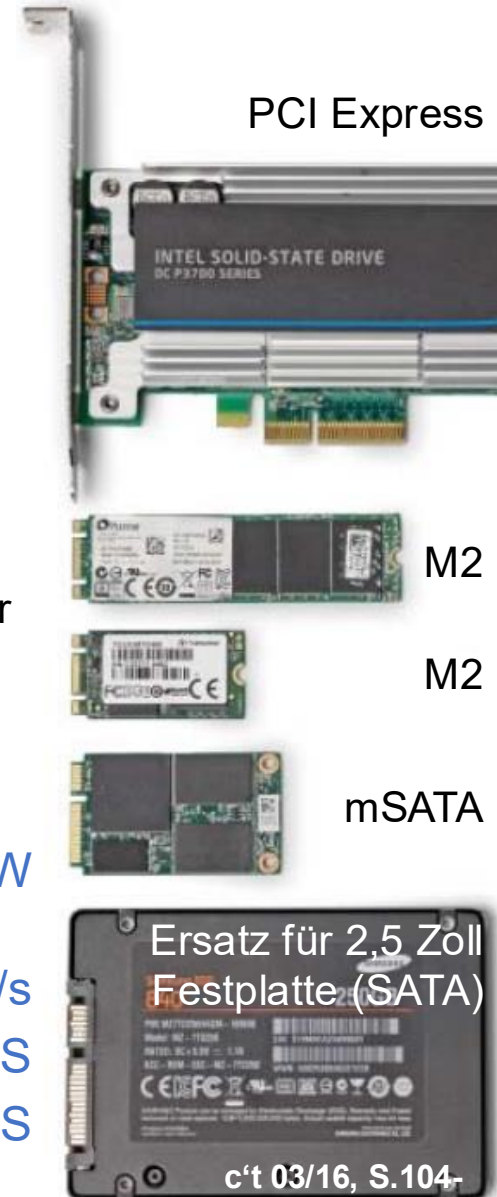
Technik und Performance

- vollelektronischer Speicher basierend auf NAND **Flash-Speicher**, keine mechanischen Komponenten (ähnlich SD-Karten)
- **Performance** für **Random Read/Write** fast wie Sequential R/W
- aus Kostengründen speichert eine Flash-Speicherzeile meist mehrere Bits (SLC -> MLC/Multi Level Cell, TLC, QLC, ...)
- **Lebensdauer** einzelner Speicherzellen **sehr gering** (ca. 1'000 (QLC) – 3'000 (TLC) – 100'000 Schreibzyklen (SLC))
- brauchbare Lebensdauer der SSD erst mit Fehlerkorrektur + Wear Leveling Algorithmen im (recht komplexen) integrierten Controller

Samsung SSD 990 PRO, 1 TB, NVMe/PCIe 4.0 x4, TLC, 150€ 2026:

- **Lebensdauer** in Terabytes Written (TBW) **600 TBW**
→ gut für Desktops, aber nicht für alle Server geeignet
- **Kontinuierliche Datenrate** (Sequential Read) **< 7500 MiB/s**
- IOPS Random Read, Blockgröße 4kB **< 22 k IOPS**
- ... 32 parallele Lese-Prozesse (Queue Depth 32) **< 1200 k IOPS**

High-Performance (\$\$\$): bis zu >3000 k IOPS für 1000 €/TB



Umgang mit der geringen Lebensdauer von Flash-Speicher

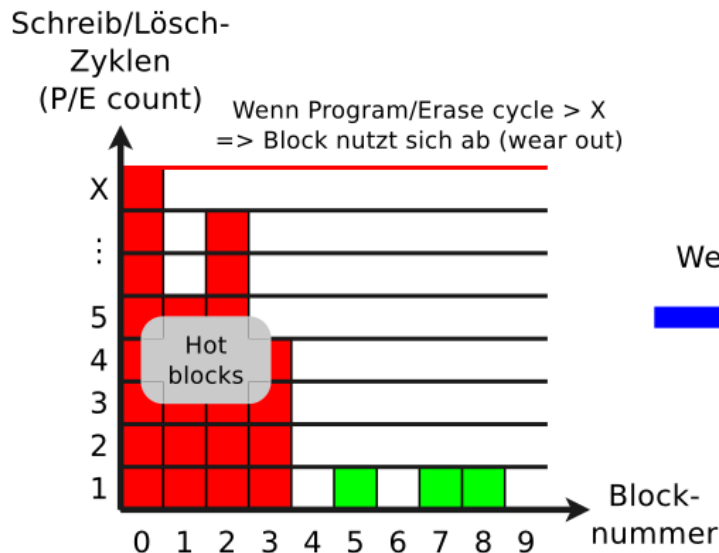
- SSD-optimierte Dateisysteme versuchen, **Schreibzugriffe zu minimieren!**
- **Wear Leveling** für gleichmäßige Abnutzung des Speichers:
 - in **SSD Controller** integriert
 - meist nicht integriert auf **SD-Karten** oder Speicher für **eingebettete Geräte**
- Spezielle **Flash File Systems** für gleichmäßige Abnutzung
 - Beispiel Linux: JFFS2, LogFS, YAFFS, F2FS
 - Speicher ist als Log organisiert, das Änderungsbefehle aufzeichnet
 - Schreibzugriffe hängen Daten an das Log an!
 - Speicher ist als zirkuläres Log strukturiert, Garbage Collection



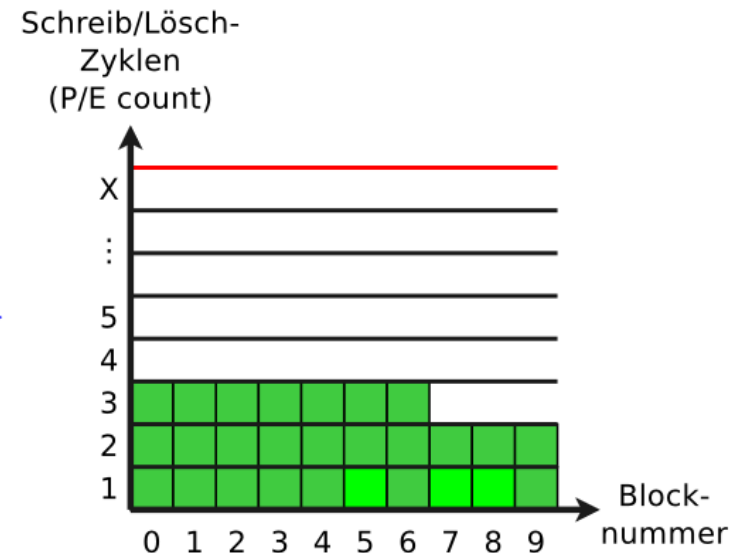
Umgang mit der geringen Lebensdauer von Flash-Speicher

Grundidee Wear Leveling

- **Zuordnungstabelle** von nach außen, also für das Betriebssystem sichtbaren Blocknummern in **interne Blocknummern**
- **Buchführung** über die Anzahl der Löschvorgänge für jeden Block
- **dynamisches Vertauschen** der internen Blöcke, so dass diese gleichmäßig abgenutzt werden



Wear Leveling



Baun: Betriebssysteme kompakt, Springer (2025).

Solid State Drive und Festplatte im Vergleich

Vorteile SSD

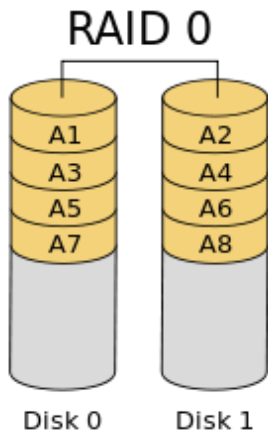
- Viel kürzere **Zugriffszeiten für Random Read** (-> IOPS Faktor 100)
 - sehr relevant, insbes. für Datenbanken oder viele parallele Prozesse
 - externe Fragmentierung der Daten nicht relevant, kein Defragmentieren, Dateisystem muss hier nicht optimieren!
- **höhere Datenrate** (Faktor 40)
- geringerer Energieverbrauch
- keine Mechanik: robust, keine Geräuschentwicklung, geringes Gewicht

Nachteile SSD

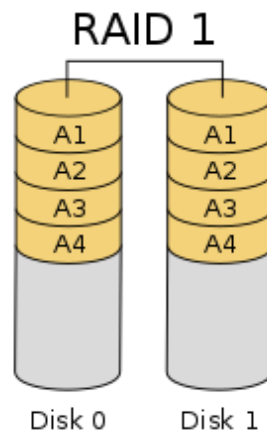
- Im Vergleich zu HDDs **hohe Kosten**
- **Eingeschränkte Anzahl Schreib-/Löschzyklen**
- Schreib-Performance kann stark einbrechen
 - Schreibzugriffe erfordern Löschen von mehrere MiB großen Blöcken!
 - idealerweise weiß die SSD, welche Blöcke frei sind und löscht diese im Voraus (TRIM)

Redundant Array of independent Disks (RAID)

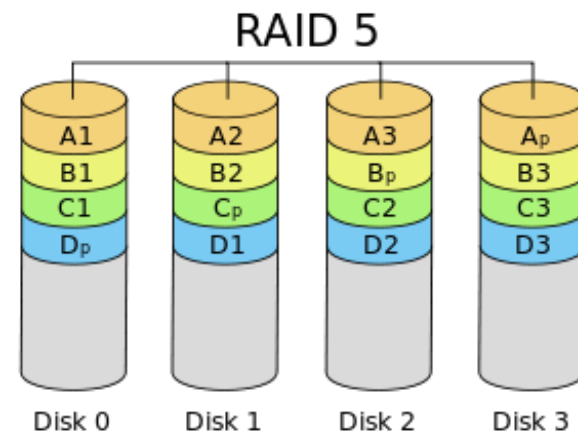
- *Performance* des Speichersystems erhöhen durch **Zusammenfassen mehrerer Speichermedien** (meist Festplatten) **zu einem großen Medium**
 - Erhöhung der **Kapazität**
 - Verbesserung der **Geschwindigkeit**
 - Verbesserung der **Datensicherheit**
- Realisierung in Hardware mit speziellen Controllern oder in Software (Linux: mdadm)
- Auswahl RAID Level:
(in der Praxis sind auch RAID-6/double parity und RAID-10/mirror+stripe sehr verbreitet)



Striping



Mirroring

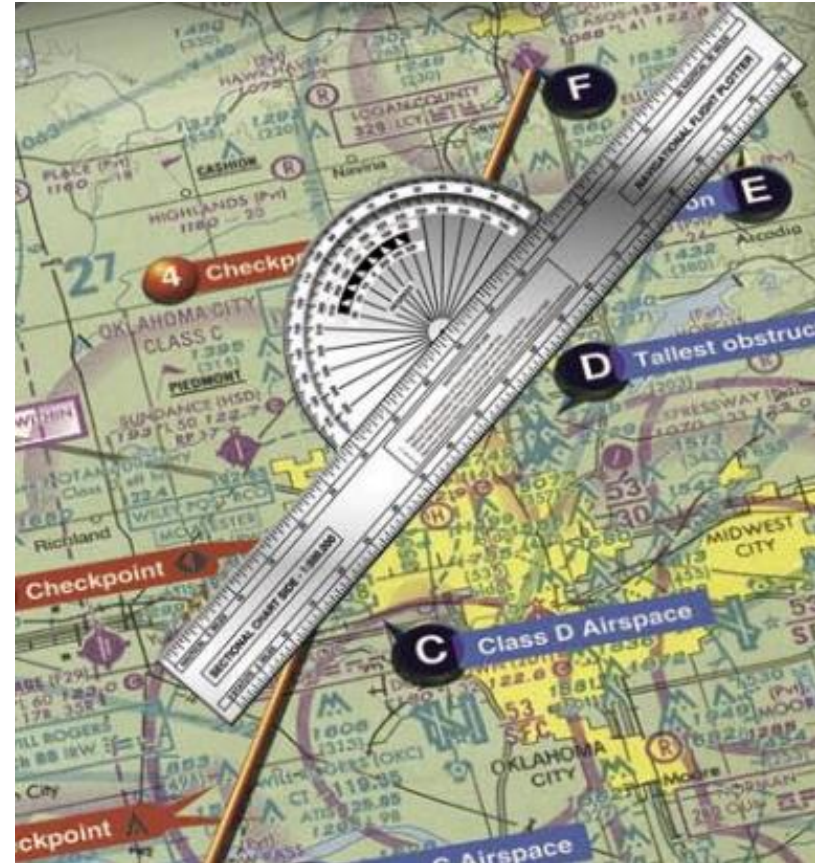


Block-Level Striping with Distributed Parity

Abb.: Wikipedia, CC BY-SA 3.0 Cburnett

Dateisysteme

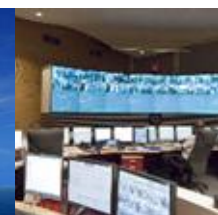
- Grundkonzepte der Implementierung
 - Ablegen von Dateien in Blöcken
 - Umgang mit freien Blöcken
 - Verwaltung von Verzeichnissen
 - Beispiele
 - FAT
 - UNIX
 - Festplatten, SSDs und RAID
- ➔ Moderne Konzepte
- Praxis-Probleme



Nur zu Lehrzwecken



Betriebssysteme



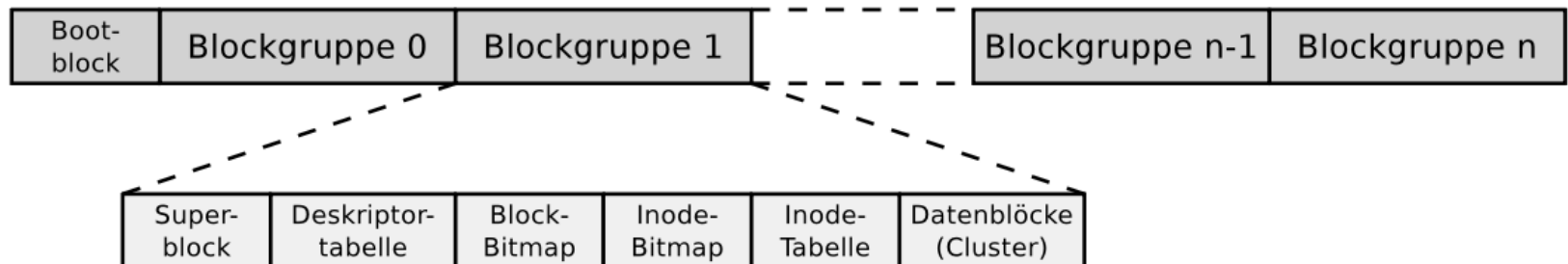
Blockgruppen in ext2/ext3/ext4

Problem

- Nutzdaten und Metadaten (I-Nodes) werden häufig gemeinsam modifiziert.
- Schlechte Performance bei Festplatten, wenn diese weit auseinanderliegen.

Lösung

- Datenträger in Blockgruppen aufteilen



- Größe in Blöcken: achtfache Blockgröße B in Bytes, z.B. 128 MiB bei B=4 kiB (Block-Bitmap mit freien Datenblöcken ist dann einen Block groß)
- Beispiele: ext2/3/4

Journaling

- Logische Schreiboperationen (`write()`) bestehen aus mehreren Schritten
- *Beispiel: Daten an eine Datei anhängen*
 1. Block-Bitmap aktualisieren
 2. I-Node aktualisieren (Datum, Dateigröße, evtl. Nummer neuer Datenblock)
 3. Falls direkte Adressierung nicht reicht: indirekten Block aktualisieren
 4. Datenblock schreiben
- *Nach jedem Schritt kann das System abstürzen!*
- Abstürze während des Schreibens führen zu **inkonsistentem Zustand**
- Dann **Konsistenzprüfung und Reparatur bei Remount/Reboot** notwendig, um weitere Datenverluste zu vermeiden
- Kann bei großen Datenträgern **sehr, sehr lange** dauern (Stunden, Tage)

Journaling

- Ziel von Journaling: **Konsistenzprüfung vereinfachen**
- Lösung:
 - Schreiboperation zuerst ins Journal eintragen ...
 - ... und dann durchführen
 - Nach Absturz die Operationen im Journal erneut durchführen
 - Operationen **idempotent** gewählt
(mehrfache Ausführung ändert Ergebnis nicht)
- Drei Varianten:
 1. Reines Metadaten Journal mit Writeback
 2. Vollständiges Journal inkl. Nutzdaten
 3. Ordered Journal als Kompromiss aus 1 und 2

Journaling

Metadaten Journal (Writeback): NTFS, XFS, Option bei ext3/4 und ReiserFS

- Journal nur über Änderungen an den Metadaten (I-Nodes), nur deren Konsistenz nach einem Absturz garantiert
- Journal sofort schreiben
- Write-Back Caching für Änderungen an regulären Blöcken
 - Schreiben des Buffer Cache nur alle 30 s (nur mit `sync()` sofort)
- Vorteil: Konsistenzprüfungen dauern nur wenige Sekunden
- Nachteil: Datenverlust durch einen Systemabsturz ist weiterhin möglich

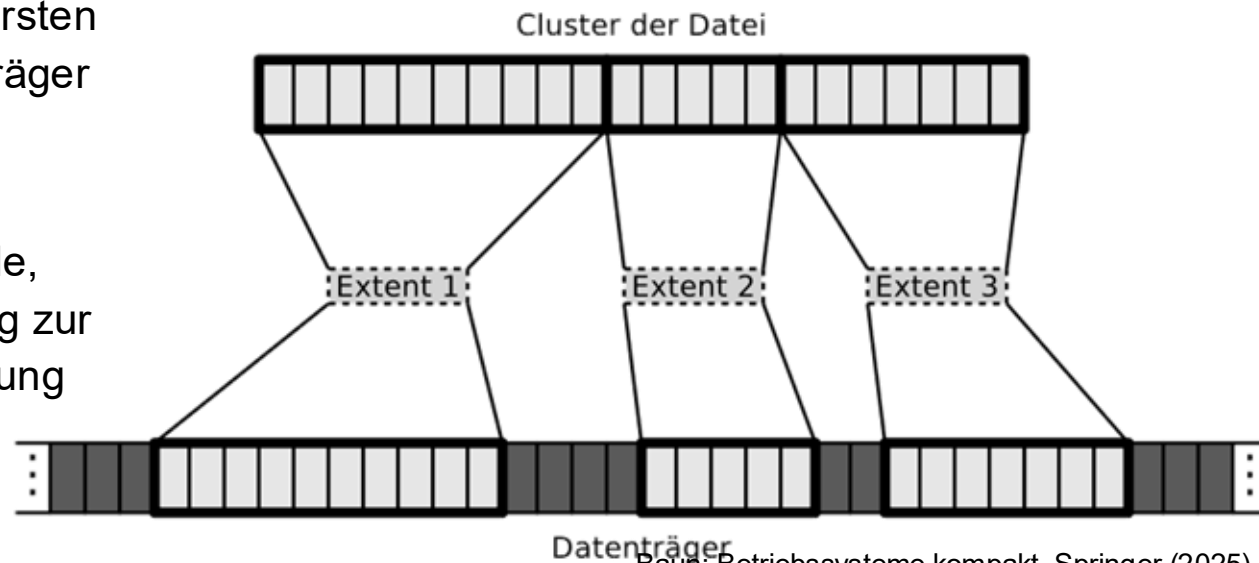
Vollständiges Journal: Option bei ext3/4 und ReiserFS

- Journal zusätzlich über Blöcke mit Dateiinhalten
- Vorteil: Auch Konsistenz der Dateiinhalte garantiert
- Nachteil: Alle Schreiboperation doppelt (Performance, Flash-Verschleiß)

Ordered Journal als Kompromiss in JFS, Default in ext3/4 und ReiserFS

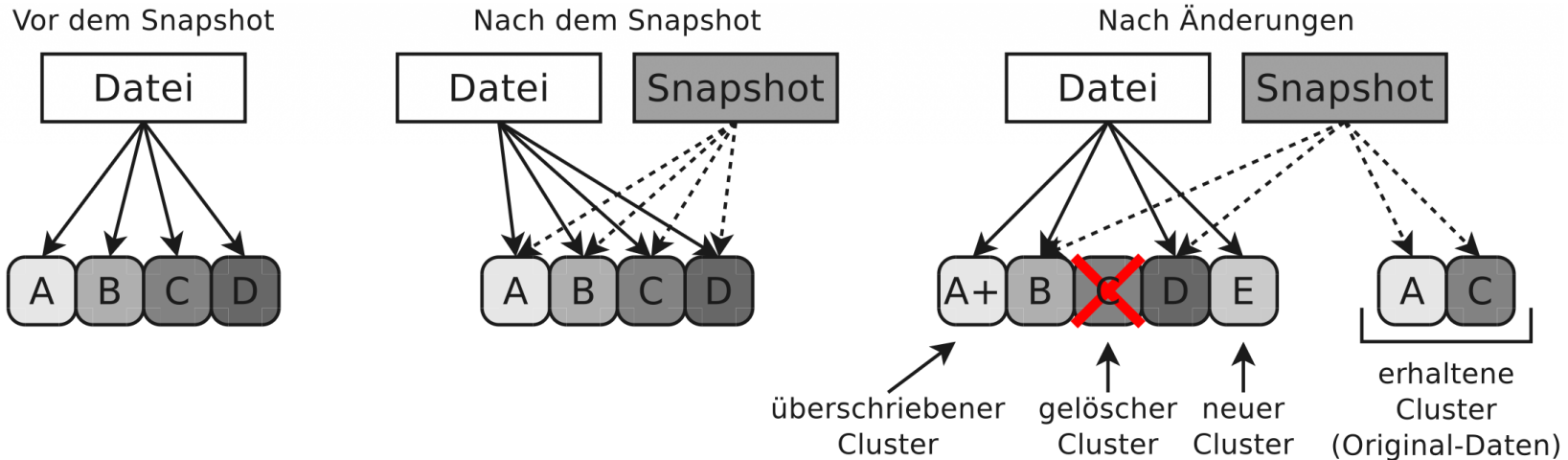
Extents

- Verwaltungsoverhead bei großen Dateien: Zusätzliche Blöcke für Blockadressen bei N-fach indirekter Adressierung benötigen viel Platz
 - Verbesserungsidee (Extents/Fragments):
 - In I-Nodes nicht mehr auf einzelne Blöcke verweisen sondern, sondern
 - auf (möglichst große) Bereiche zusammenhängender Blöcke (Extents)
 - Nummer des Extent-Startblockes innerhalb der Datei
 - Größe des Extents (Anzahl Blöcke)
 - Blocknummer des ersten Clusters auf Datenträger
- ➔ weniger Verwaltungs-overhead
- Extent-Info zuerst in Inode, dann Extent-Baum analog zur indirekten Blockadressierung
 - Beispiele: JFS, XFS, btrfs, NTFS, ext4



Baum: Betriebssysteme kompakt, Springer (2025).

Copy-On-Write



- **Snapshots** manuell erzeugen zum Einfrieren einer **Version** des Dateisystems
- Snapshot enthält Zeiger auf Cluster mit Dateiinhalten
- **copy-on-write**: erst Schreibzugriffe erzeugen Kopien der betroffenen Cluster
- Ältere Dateiversionen aus Snapshot können so wiederhergestellt werden
- Snapshots können **sehr schnell** erzeugt werden und kosten **wenig Platz**
- Beispiele: ZFS, btrfs und ReFS (Resilient File System, neu in MS Windows)



Log-Structured File Systems für Flash-Speicher

Problem: In-Place-Update auf Flash

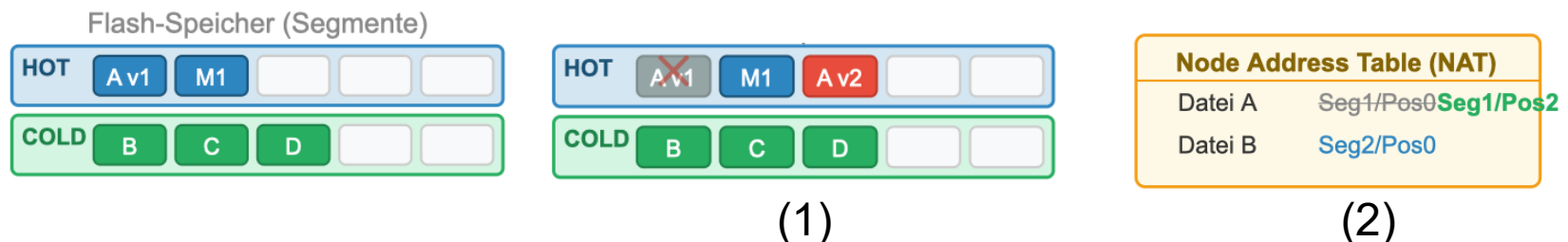
- Flash kann **nur in großen Blöcken (~MiB) gelöscht** werden (Bit auf 0 setzen)
- Schreiben nur in Seiten (~4-128 kiB)** (Bits auf 1 setzen)
- Worst Case Änderung 1 Byte: ganzen Löschblock lesen, löschen, zurückschreiben

Lösung: Log-Structured FS

- Bei Änderung die alten Daten **nicht** überschreiben.
- Geänderte Daten **sequentiell ans Ende eines Logs anhängen (append-only)**
- Veraltete Log Records **später** durch **Garbage Collection** löschen und zurückgewinnen

Beispiel: Flash-Friendly File System F2FS (Samsung 2012, heute Standard in Android)

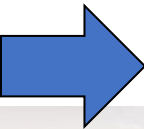
- Trennung von heißen (Metadaten, oft geändert) und kalten (selten geändert) Daten
- Node Address Table (NAT): entkoppelt logische Blocknummer von physischer Position
- Beispiel A ändern: (1) v2 von A anhängen, (2) NAT aktualisieren, (3) Garbage Collection



Dateisysteme

- Grundkonzepte der Implementierung
 - Ablegen von Dateien in Blöcken
 - Umgang mit freien Blöcken
 - Verwaltung von Verzeichnissen
- Beispiele
 - FAT
 - UNIX
- Festplatten, SSDs und RAID
- Moderne Konzepte

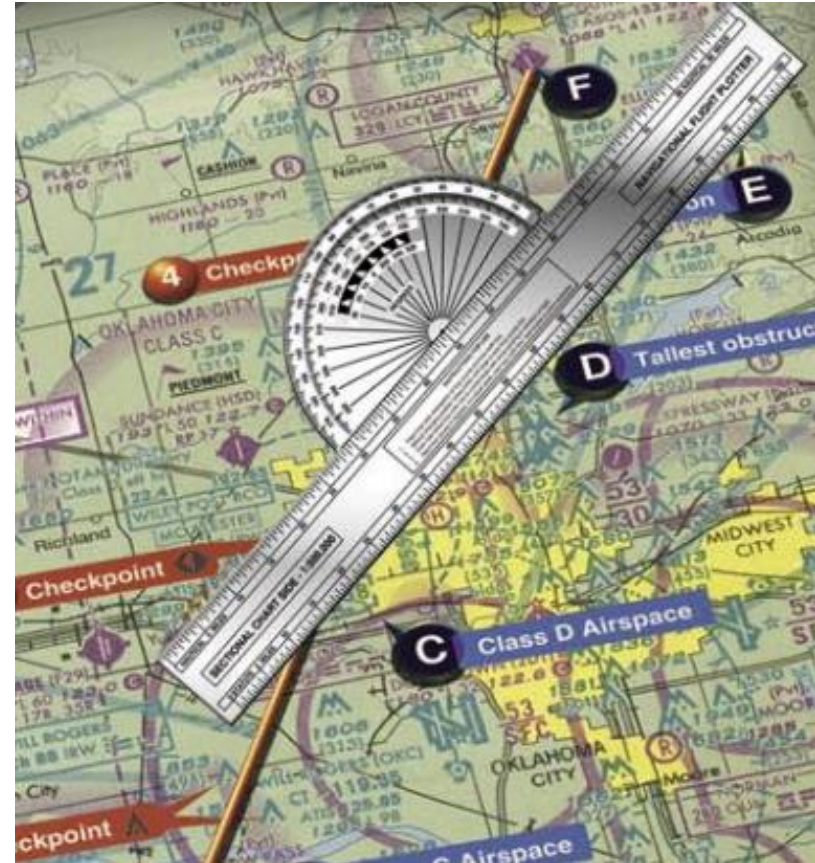
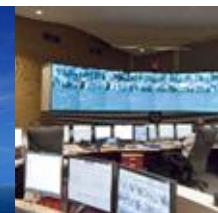
Praxis-Probleme



Nur zu Lehrzwecken



Betriebssysteme



Partitionen alt: Klassischer PC/MBR

- Eine Festplatte ist in **Partitionen** (Bereiche von Sektoren mit LBA-Adressen) organisiert, jede Partition hat ein eigenes Dateisystem.
- **MBR Partitionsschema** (x86-Rechner)
 - Zu Beginn der Festplatte befindet sich der **Master Boot Record** (MBR), der Teile des Bootloaders und die Partitionstabelle enthält.
 - Eine **Partition** wird als **aktiv** deklariert, von ihr bootet ein Rechner
- Beispiel für ein mögliches Layout eines Dateisystems (klassischer PC):

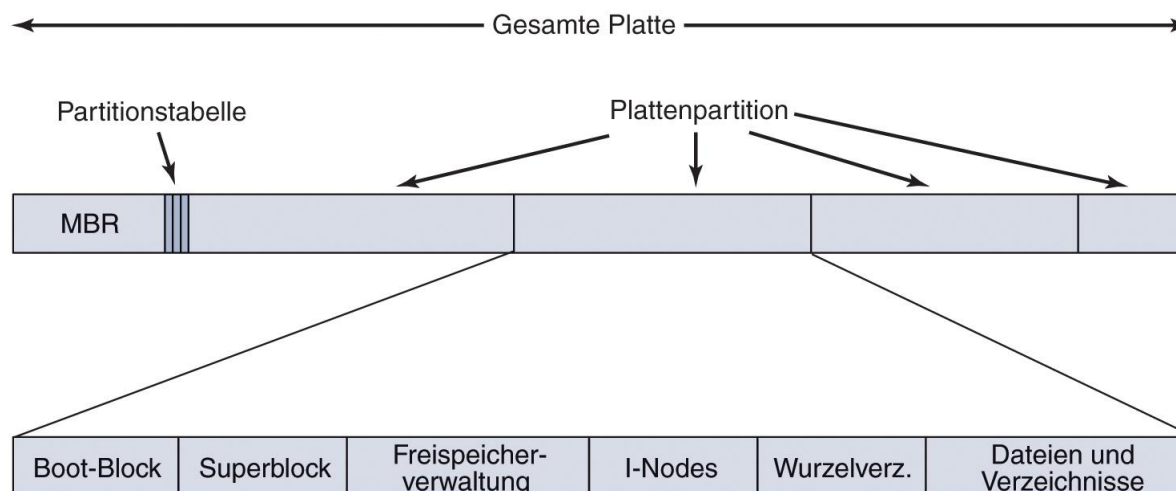
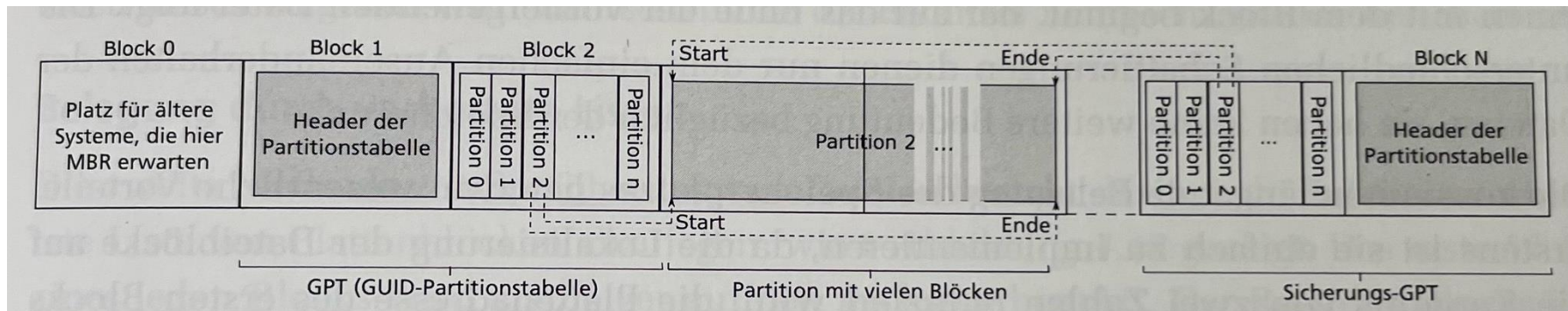


Abb.: Tanenbaum



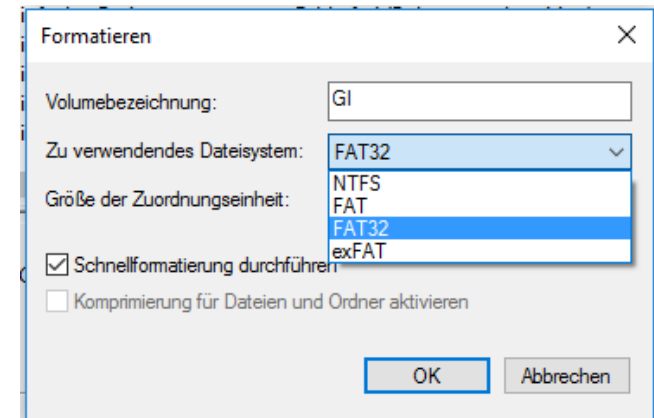
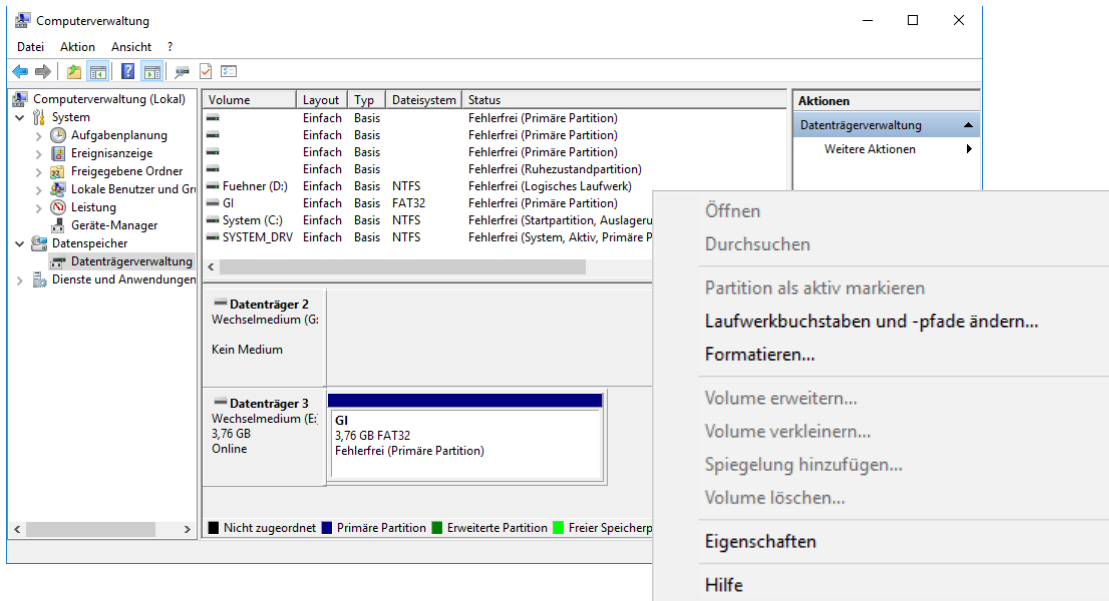
Partitionen neu: Moderner PC/UEFI

- Modernere PC verwenden UEFI (Unified Extensible Firmware Interface) Firmware
- kein **MBR mit Bootloader** mehr erforderlich, sondern echtes Dateisystem (meist FAT) mit Bootprogramm auf **EFI-Systempartition (ESP)** im **Portable Executable (PE)** Format
- Auch hier werden Speichermedien partitioniert, Partitionstabelle im **GUID Partition Table (GPT)** Format
- Festplattenlayout:



Formatieren

- Ein neuer Datenträger muss partitioniert werden
- Die logischen Partitionen müssen dann formatiert werden
- Beim Formatieren wird ein leeres Dateisystem vom gewünschten Typ und mit den gewünschten Parametern angelegt
- Vorsicht: Beim Formatieren werden alle Daten in der Partition gelöscht!
- Linux: `mkdosfs /dev/sdXX -v -S 512 -s 2 -F 16 -n Daten`



Dateisysteme nutzen unter Linux

`mount [-t typ] <Gerät> <Mount-Point>`

- Hängt Dateisystem auf Gerät an der Stelle Mount-Point in den Verzeichnisbaum ein
- Beispiel für Gerät: erste Partition der ersten SSD `/dev/sda1`
- `mount` ohne Parameter zeigt aktuelle Mounts an

`/etc/fstab`

- Konfigurationsdatei für automatisches Mounten bei Systemstart

`umount <Mount-Point>`

- Hängt das Dateisystem wieder aus
- Schlägt fehl, wenn noch Dateien auf dem Dateisystem geöffnet
- `lsuf` / `fuser` zeigen an, wer Dateien nutzt / geöffnet hat

`fdisk` / `parted`

- Partitionstabellen anzeigen / editieren (erfordert Superuser-Rechte)

Integritätstest & Reparatur

Das Problem

1. Sektoren auf dem Speichermedium können **physisch beschädigt** werden oder aufgrund von **Alterung** nicht mehr lesbar sein
2. Verwaltungsinformation kann beim Abschalten des Rechners **ohne Herunterfahren** nur teilweise geschrieben worden sein -> **Inkonsistenzen**

Integrität prüfen & Fehler reparieren

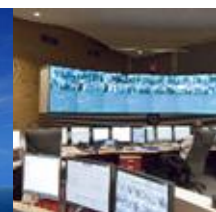
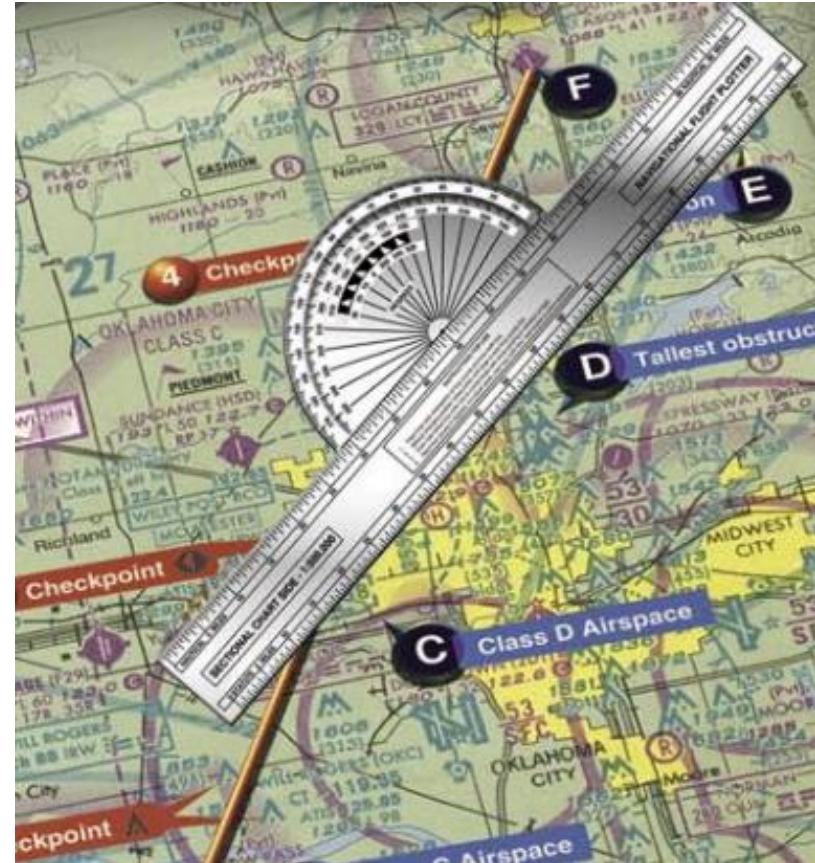
- **Datenverlust** lässt sich nicht immer vermeiden
- **Windows:** Auf Laufwerk klicken, dann Eigenschaften/Tools-Tab/Jetzt prüfen
- Windows-Kommandozeile: **chkdsk**
- **Linux-Tool** abhängig vom Dateisystem, z.B. ext4: **e2fsck -n /dev/sda2**
- Dateisystem vorher aushängen (umount)
- Automatischer Aufruf beim Systemstart, wenn das Medium beim Herunterfahren nicht mit umount entfernt wurde

Defragmentieren von HDDs

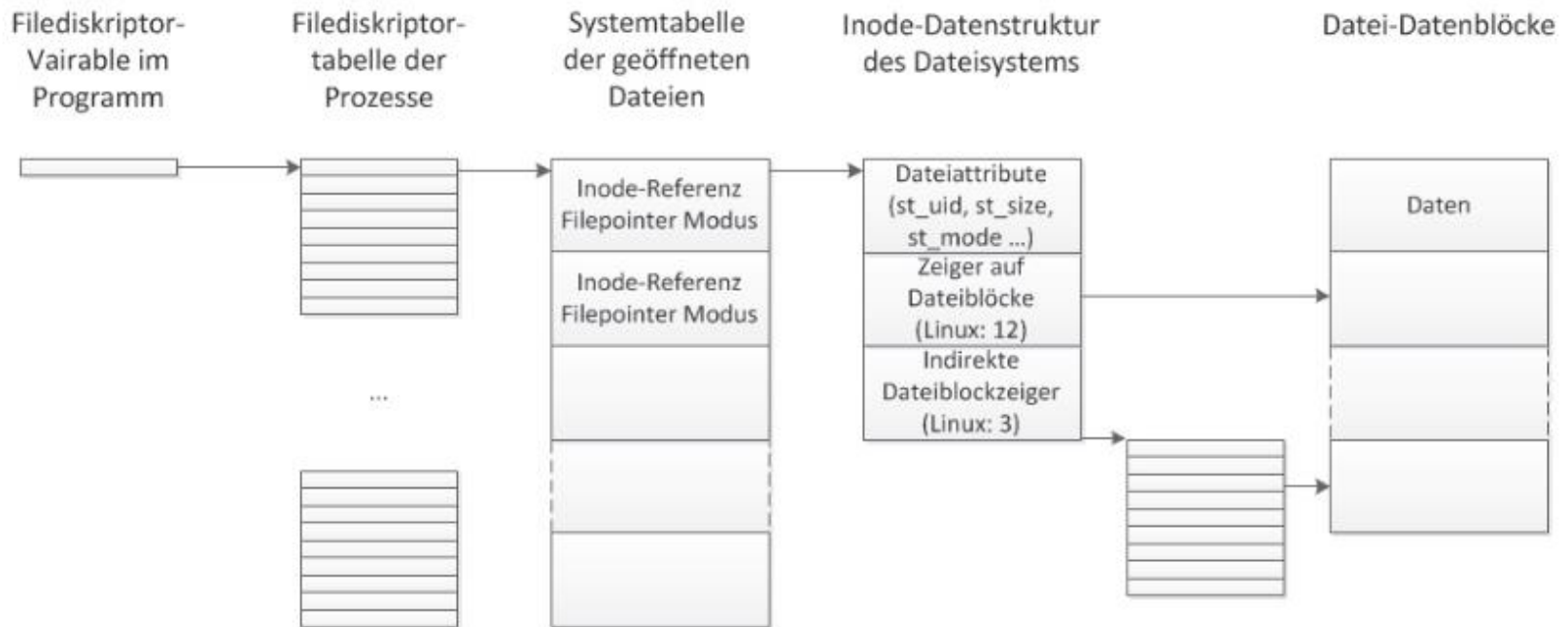
- Externe Fragmentierung von Dateien führt dazu, dass in der Datei aufeinanderfolgende Inhalte auf der Festplatte dann räumlich getrennt werden
- Wenn die Datei fortlaufen gelesen wird, werden für HDDs **sehr** zeitaufwändige Bewegungen des Schreib-/Lesekopfes notwendig, um entfernte Cluster zu erreichen
- Beim Defragmentieren wird die Zuordnung der Cluster zu Dateien reorganisiert:
 1. In **Dateien** aufeinanderfolgende Daten in aufeinanderfolgenden Sektoren speichern
→ Performancegewinn **beim fortlaufenden Lesen**
 2. **Freien Speicher** in zusammenhängenden Bereich organisieren, damit neue Dateien in aufeinanderfolgenden Sektoren geschrieben werden können
→ Performancegewinn **beim fortlaufenden Schreiben** (und danach Lesen)
- Beide Maßnahmen reduzieren zeitaufwändige Kopfbewegungen
- Für Unix-Systeme mit vielen Prozessen weniger relevant, das fortlaufendes Lesen selten
- Bei **SSDs** nicht nützlich, da IOPS für zufällige Zugriffsmuster nicht schlechter als fortlaufend (und wegen der Schreibzugriffe sogar **schädlich** für deren Lebensdauer!)

Dateisysteme

- Grundkonzepte der Implementierung
 - Ablegen von Dateien in Blöcken
 - Umgang mit freien Blöcken
 - Verwaltung von Verzeichnissen
- Beispiele
 - FAT
 - UNIX
- Festplatten, SSDs und RAID
- Moderne Konzepte
- Praxis-Probleme



Vom Filedescriptor zum Datenblock

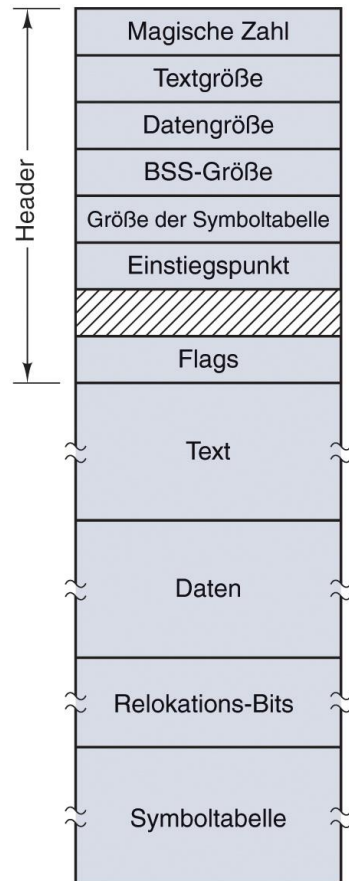


- Eine Filedescriptortabelle pro Prozess; diese verweist auf Systemtabelle für Filedescriptoren, die wiederum auf Inodes und Datenblöcke verweist
- Systemweite Tabelle enthält Position für Schreib-/Lesezugriffe in Datei
- Systemweite Tabelle notwendig, damit sich Kindprozesse die gleiche Position teilen können

Ehse et al.: Systemprogrammierung in UNIX/Linux, Springer (2012).

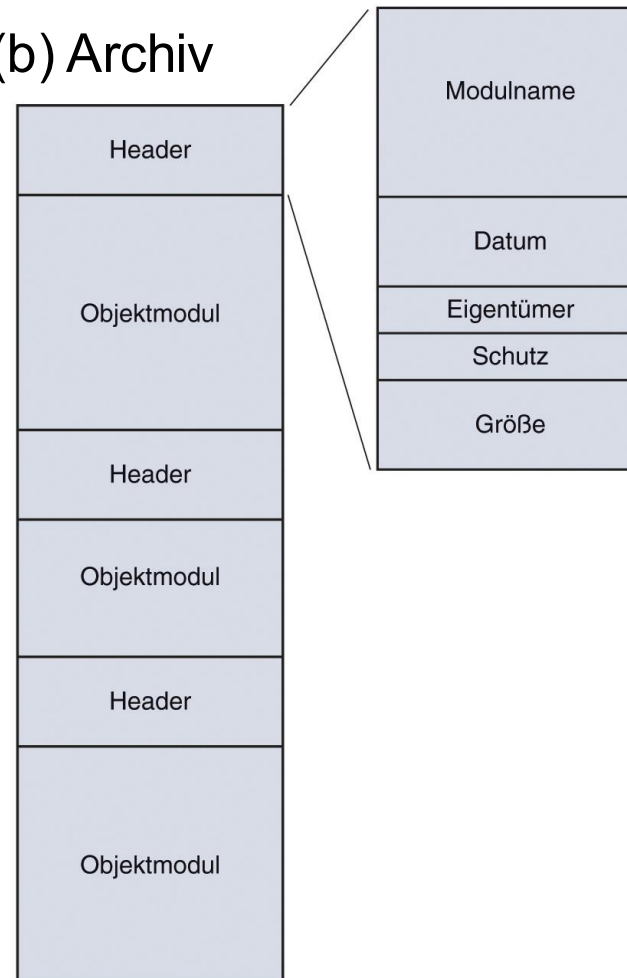
Aufbau und Identifizierung binärer Dateien

(a) ausführbare Datei



a

(b) Archiv



b

Dateiattribute

Attribute	Bedeutung
Schutz	Wer kann wie auf die Datei zugreifen?
Passwort	Passwort für den Zugriff auf die Datei
Urheber	ID der Person, die die Datei erzeugt hat
Eigentümer	Aktueller Eigentümer
Read-only-Flag	0: Lesen/Schreiben; 1: nur Lesen
Hidden-Flag	0: normal; 1: in Listen nicht sichtbar

Dateiattribute

Attribute	Bedeutung
System-Flag	0: normale Datei; 1: Systemdatei
Archiv-Flag	0: wurde gesichert; 1: muss noch gesichert werden
ASCII/Binär-Flag	0: ASCII-Datei; 1: Binärdatei
Random-Access-Flag	0: nur sequenzieller Zugriff; 1: wahlfreier Zugriff
Temporary-Flag	0: normal; 1: Datei bei Prozessende löschen
Sperr-Flags	0: nicht gesperrt; nicht null: gesperrt
Datensatzlänge	Anzahl der Bytes in einem Datensatz
Schlüsselposition	Offset des Schlüssels innerhalb des Datensatzes
Schlüssellänge	Anzahl der Bytes im Schlüsselfeld
Erstellungszeit	Datum und Zeitpunkt der Dateierstellung
Zeitpunkt des letzten Zugriffs	Datum und Zeitpunkt des letzten Zugriffs
Zeitpunkt der letzten Änderung	Datum und Zeitpunkt der letzten Dateiänderung
Aktuelle Größe	Anzahl der Bytes in der Datei
Maximale Größe	Anzahl der Bytes für maximale Größe der Datei

Master Boot Record (MBR)

- Erster Block auf einem partitionsfähigen Medium

Structure of a master boot record					
Address			Description		Size in bytes
Hex	Oct	Dec			
0000	0000	0	code area		440 (max. 446)
01B8	0670	440	disk signature (optional)		4
01BC	0674	444	Usually nulls; 0x0000		2
01BE	0676	446	Table of primary partitions (Four 16-byte entries, IBM partition table scheme)		64
01FE	0776	510	55h	MBR signature; 0xAA55	2
01FF	0777	511	AAh		
MBR, total size: 446 + 64 + 2 =					512

Code für Teil des Bootvorgangs (Bestimmung des aktiven Partition)

32 Bit Disk-Signatur

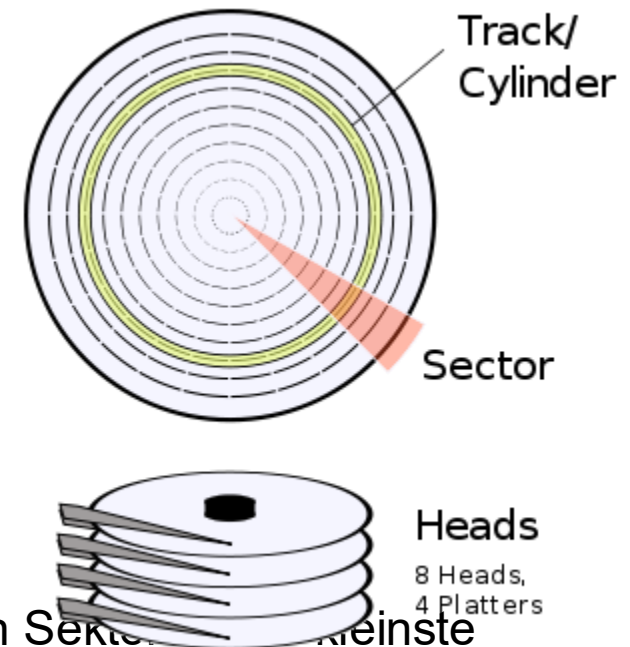
Partitionstabelle

Magic Number

- MBR unter Linux auslesen:
`dd if=/dev/hda of=mbr.dmp bs=512 count=1`

Cylinder Head Sector (CHS)

- Frühere Methode zur Adressierung der Blöcke auf einer Festplatte
- **Heads** – der Kopf schreibt und liest die Daten von der Magnetplatte, oft werden zwei Köpfe (einer für jede Seite) pro Platte eingesetzt.
- **Cylinder** – je nach Position des Kopfs wird ein unterschiedlicher **Tracks** gelesen. Durch die Rotation des Plattenstapels entsteht räumlich gesehen ein **Zylinder** wenn wir mehrere Köpfe betrachten.
- **Sector** – Tracks werden in Sektoren unterteilt, ein Sektor ist die kleinste Speichereinheit auf der Festplatte, typischerweise 512 Byte.
- **Block** (Unix) bzw. **Cluster** (Windows) – bezeichnet ein Vielfaches eines Sektors, d.h. ein Vielfaches von 512 Byte.



Logical Block Addressing (LBA)

- CHS setzt eine bestimmte Festplattengeometrie voraus
- Neuere Festplatten speichern auf den äußeren Tracks mehr Sektoren (Constant Angular Velocity).
- Jeder Block (entspricht Sektor im CHS Adressierung) erhält nach der LBA Adressierung *eine* Nummer, die Zählung beginnt bei 0. Die Nummerierung erfolgt linear.
- Es gibt zwei Adressformate
 - 28 Bit = 128 GiB (bei 512 Byte Block/Sektorgröße)
 - 48 Bit = 128 PiB
- CHS Tupel (cylinder/head/sector) können mit folgender Formel auf LBA Adressen umgerechnet werden

$$\text{LBA} = ((C * \text{HPC}) + H) * \text{SPT} + S - 1$$

MBR - Partitionstabelle

4 Einträge (= 4 Partitionen) mit folgenden Angaben

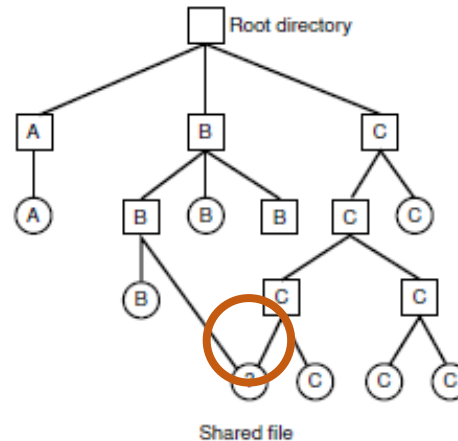
Offset	Length (Byte)	Description
0x00	1	status (0x80 = bootable (<i>active</i>), 0x00 = non-bootable, other = invalid)
0x01	3	CHS address of first absolute sector in partition
0x04	1	partition type (type of file system)
0x05	3	CHS address of last absolute sector in partition
0x08	4	LBA of first absolute sector in the partition
0x0C	4	Number of sectors in partition

ID	Developer	Description
0x00	??	Empty partition
0x01	Microsoft	FAT 12

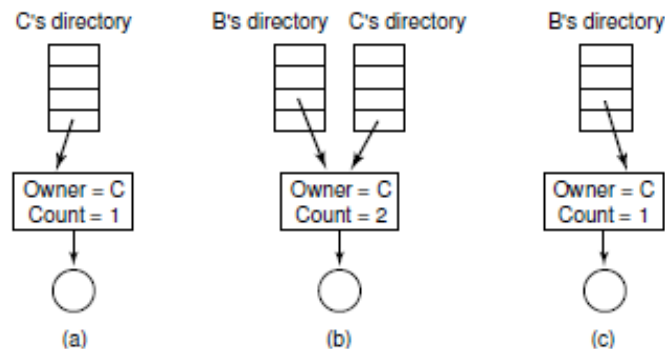
Sektorgröße 2^9 (= 512) Bytes → weder die maximale Größe einer Partition noch deren Startadresse kann $2^{32} \times 2^9 = 2^{41}$ Bytes = 2×2^{40} , d.h. 2 TiB übersteigen.

Gemeinsam benutzte Dateien / Harte Links

Das Ziel:



Lösung 1:



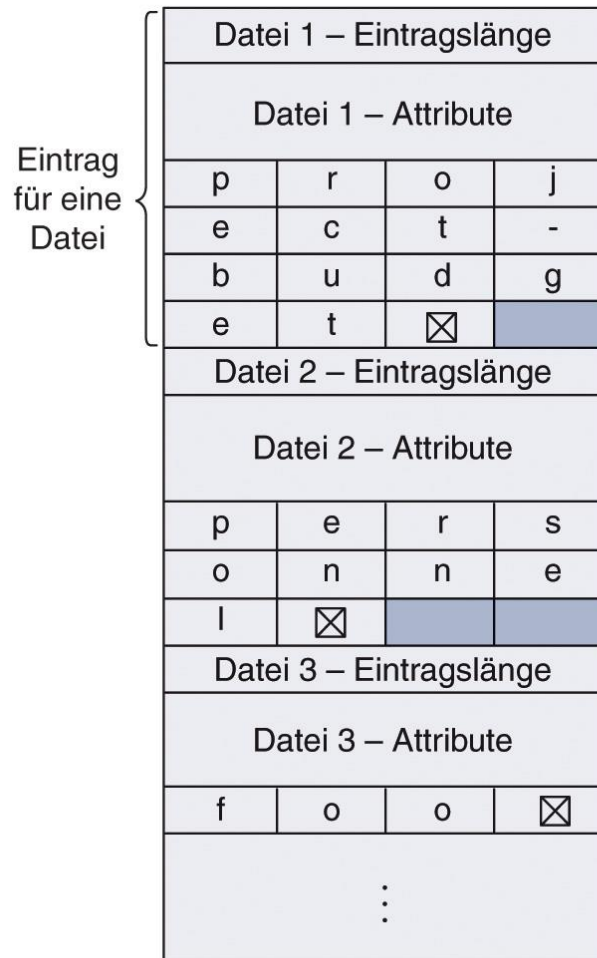
(a) C erzeugt Datei, (b) B linkt Datei, (c) C löscht Datei

Lösung 2: B erzeugt Datei vom Typ LINK, die einen Pfad enthält

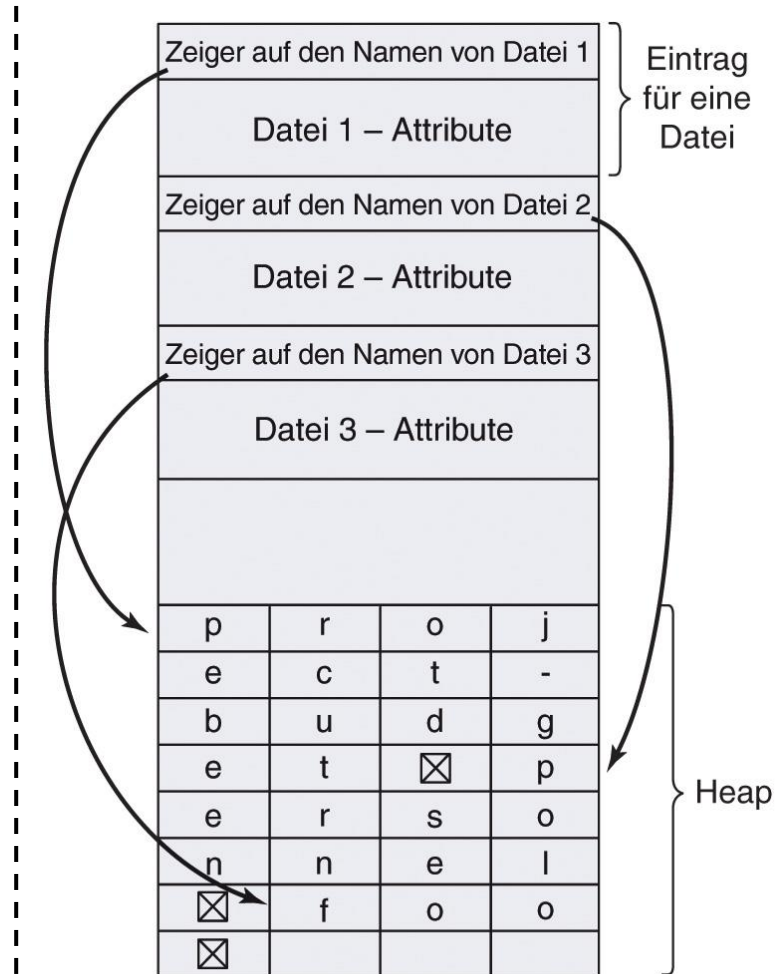
UNIX: Harte und symbolische Links

- Harte Links (Lösung 1)
 - Zusätzliche *Einträge* in Verzeichnissen
 - werden mit `ln` erzeugt
 - Linkcounter kann mit `ls -l` angezeigt werden
 - `rm` löscht einen Eintrag im Verzeichnis
 - Dateien werden erst gelöscht wenn der Linkcounter 0 ist
- Symbolische Links (Lösung 2)
 - Eine spezielle Form einer *Datei*
 - werden mit `ln -s` erzeugt

Lange Dateinamen (>8.3) unter Windows



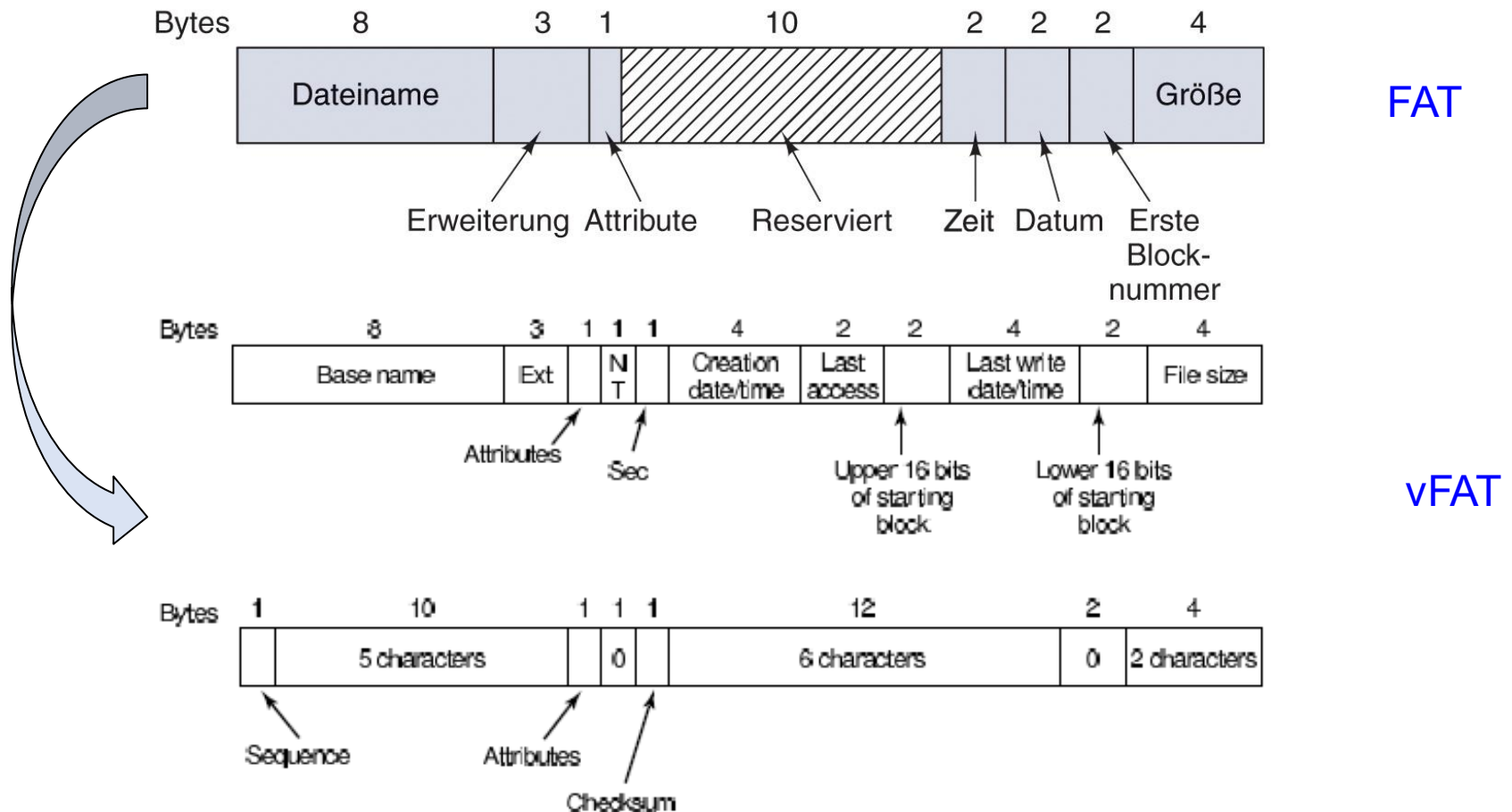
In-Line (z.B. FAT)



Im Heap

Lange Dateinamen unter Windows 95 ff

Eintrag für einen Teil eines langen Dateinamens (32 Byte),
markiert durch *Attribute=0x0F*



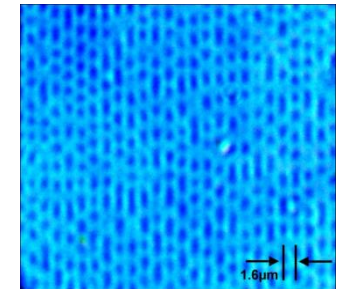
Beispiel für einen langen Dateinamen

The quick brown fox jumps over the lazy dog

	68	d o g				A	0	C	K					0					
	3	o v e r				A	0	C	K	t h e l a				0	z y				
	2	w n f o				A	0	C	K	x j u m p				0	s				
	1	T h e q				A	0	C	K	u i c k b				0	r o				
	T	H E Q U I ~ 1				A	N	T	S	Creation time		Last acc	Upp	Last write		Low	Size		
Bytes																			

ISO-9660 Dateisystem (CD-ROM)

- Eine CD besteht aus einer *Spiralspur* von Bits („Pits“ und „Lands“) mit einer nutzbaren Blockgröße von 2k
- CD-ROM wurde 1988 als ISO 9660 standardisiert
- Einschränkung durch Unterstützung von MS-DOS (8.3 Dateinamen)
- Nur 8 Verzeichnisebenen
- Erweiterung durch Rock-Ridge (UNIX) und Joliet (Windows)
- ISO-9660 Verzeichniseintrag (variable Länge):



- Dateien werden **sequentiell** (als zusammenhängende Menge von Blöcken) abgelegt.